BACHELOR 3

INFRAREADY

Administration systèmes Linux & Windows

01 **Dossier technique**
Architecture, configurations et procédures

02 **Preuves de fonctionnement**
Captures commentées et matrice de validation

03 **Rapport d'incident**
Perte de collecte Node Exporter et rétablissement

Ilies Bossuyt & Léo Collowald

Administration systèmes Linux / Windows

DÉPÔT GITHUB

github.com/Leocollowald/Projet-InfraReady

NAVIGATION

Sommaire & guide de lecture

I. Dossier technique

Contexte, architecture et inventaire 3

Installation, réseau et DNS 6

Comptes et administration Windows 10

Services, stockage et droits Linux 12

Automatisation avec Ansible 18

Maintenance Linux et Windows 25

Supervision Prometheus / Grafana 29

Dépôt GitHub et intégration continue 33

Sécurité et limites connues 35

II. Preuves de fonctionnement

Matrice de validation - infrastructure et administration 36

Matrice de validation - automatisation et exploitation 37

III. Rapport d'incident

Contexte, chronologie et cause 38

Détection, correction et retour à la normale 39

Bilan et actions préventives 41

01. Besoin, périmètre et méthode

InfraReady est une entreprise fictive dont les serveurs Linux et Windows sont administrés de façon hétérogène. Le projet vise à standardiser leur configuration, rendre les opérations répétables et réduire le temps nécessaire au diagnostic d'un incident.

Nous avons mis en place un laboratoire de trois machines virtuelles dans VMware. Le serveur Windows fournit l'annuaire et le DNS, SERVICE1 héberge les services Linux, OPS1 centralise l'automatisation et la supervision.

Objectif du sujet	Réalisation documentée
Installer et relier les serveurs	Trois VM, adresses fixes, réseau NAT VMnet8.
Administrer Windows et Linux	AD DS, comptes et groupes, droits NTFS, Nginx, permissions Linux et stockage ext4.
Rendre les opérations reproductibles	Playbooks Nginx, Windows et maintenance Linux.
Maintenir et superviser	Maintenance contrôlée, métriques Linux/Windows et tableau de bord Grafana.
Valider et diagnostiquer	Workflow GitHub Actions et incident réversible sur Node Exporter.

Documents réunis

Ce PDF rassemble le dossier technique, les preuves de fonctionnement et le rapport d'incident. Le dépôt GitHub constitue le livrable de code.

02. Architecture du laboratoire

Les trois machines partagent le même réseau virtuel. Les rôles restent séparés afin de distinguer les services d'infrastructure, le serveur applicatif et le poste d'exploitation. Les opérations d'administration et de collecte sont initiées depuis OPS1.

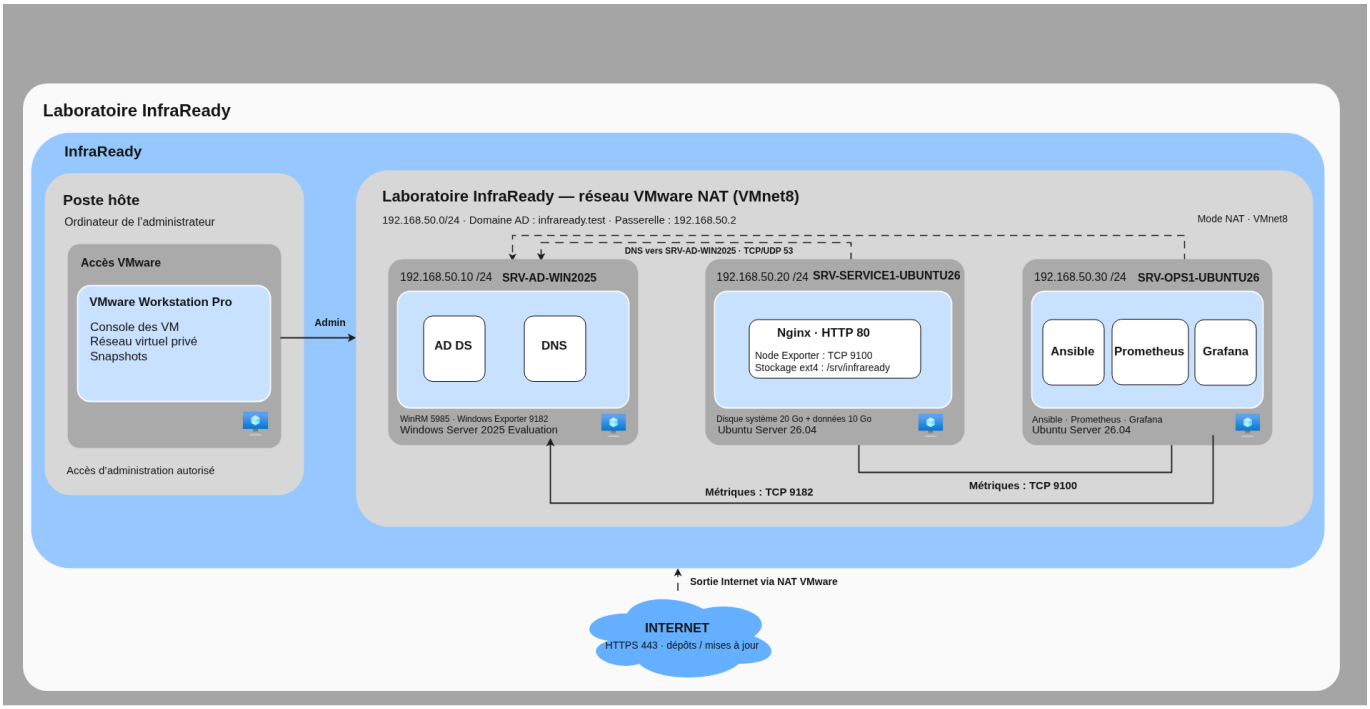


Schéma de synthèse redessiné d'après les deux fichiers draw.io fournis. Les flèches représentent les connexions initiées par OPS1.

Flux principal	Destination / port	Fonction
OPS1 vers SERVICE1	TCP 22 ; TCP 9100	Administration SSH ; collecte Linux.
OPS1 vers AD	TCP 5985 ; TCP 9182	WinRM/Kerberos ; collecte Windows.
SERVICE1 / OPS1 vers AD	TCP/UDP 53 ; Kerberos 88	Résolution DNS ; authentification Kerberos depuis OPS1.
Poste d'administration vers SERVICE1 / OPS1	TCP 80 ; TCP 3000	Test Nginx ; consultation Grafana.

03. Inventaire et adressage

Machine / rôle	Système et stockage	Adresse fixe
SRV-AD-WIN2025 Annuaire, DNS, exporter Windows	Windows Server 2025 Évaluation Disque système : 50 Go	192.168.50.10/24
SRV-SERVICE1-UBUNTU26 Nginx, comptes, données, exporter Linux	Ubuntu Server 26.04 Système : 20 Go Données : 10 Go	192.168.50.20/24
SRV-OPS1-UBUNTU26 Ansible, Prometheus, Grafana	Ubuntu Server 26.04 Disque système : 20 Go	192.168.50.30/24

Paramètre commun	Valeur retenue	Utilité
Réseau / masque	192.168.50.0/24 255.255.255.0	Communication directe entre les VM.
Passerelle	192.168.50.2	Sortie du réseau privé via VMware NAT.
DNS des clients	192.168.50.10	Résolution du domaine et du contrôleur AD.
Domaine	infraready.test	Namespace DNS / domaine Kerberos.
FQDN du contrôleur	srv-ad-win2025.infraready.test	Nom utilisé pour WinRM avec Kerberos.

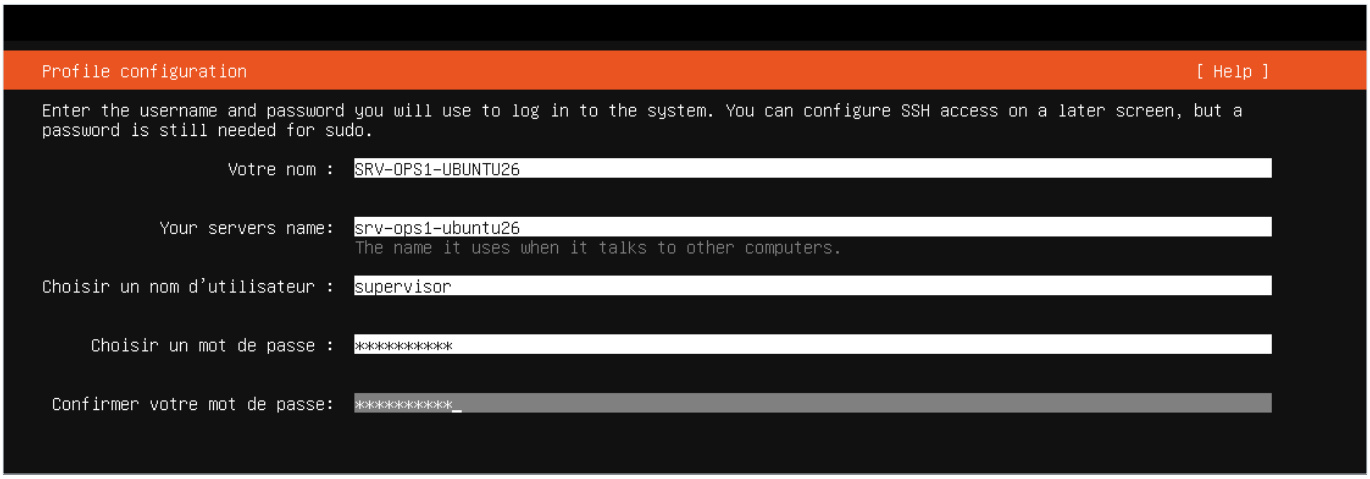
Disque virtuel et volume logique
Sur SERVICE1, le disque système de 20 Go contient notamment un volume logique racine de 10 Go. Il ne faut donc pas confondre capacité du disque virtuel et espace disponible sur /. Le disque de données de 10 Go est séparé.

04. Installation et nommage

La mise en place commence par la création de trois VM dans VMware, leur raccordement au même réseau VMnet8 et l'installation des systèmes. Des noms distincts identifient le rôle de chaque machine et facilitent la lecture des inventaires, journaux et métriques.



F01. Profil d'installation Ubuntu du serveur SERVICE1 : nom d'hôte et compte supervisor. [SERVICE, p. 1]



F02. Profil d'installation Ubuntu du contrôleur OPS1. [OPS1, p. 1]

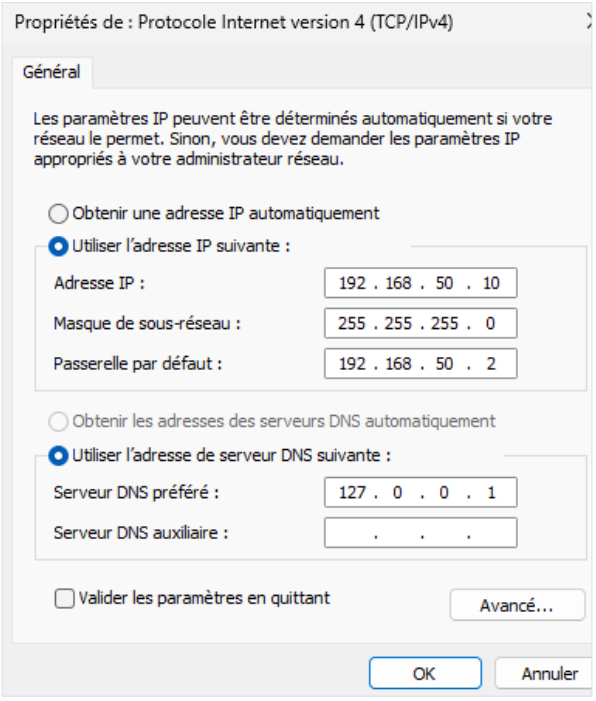
Ordre de préparation

- Vérifier les ressources du poste hôte, les disques virtuels et l'absence de conflit avec le réseau 192.168.50.0/24.
- Installer Windows et les deux Ubuntu, puis appliquer les adresses fixes et la passerelle NAT.
- Installer AD DS/DNS sur Windows.
- Tester les noms et les communications avant d'installer Ansible et la supervision.

Le compte local **supervisor** est utilisé pour l'administration des deux Ubuntu.

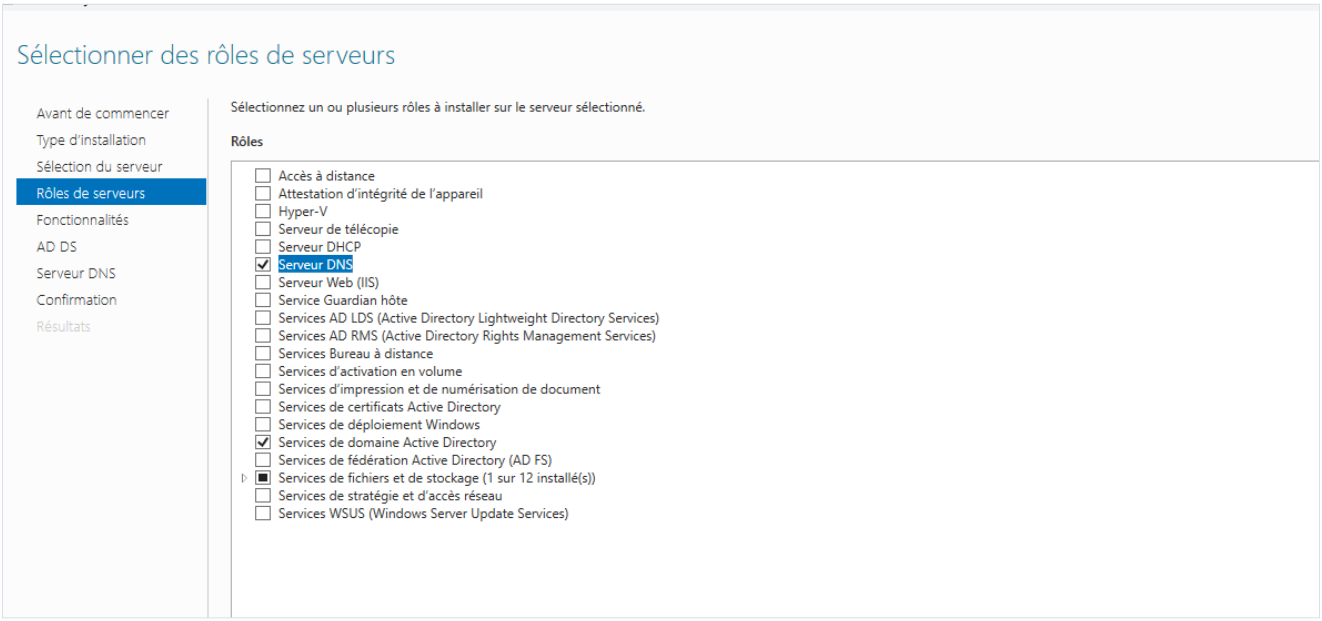
05. Adresse Windows et rôles AD DS

Le contrôleur reçoit l'adresse 192.168.50.10, un masque /24 et la passerelle 192.168.50.2. Son client DNS utilise la boucle locale 127.0.0.1 : les requêtes de cette VM sont ainsi dirigées vers son propre service DNS.



F03. Paramètres IPv4 du contrôleur de domaine : IP fixe, passerelle et DNS local. [AD, p. 2]

Les rôles **Services de domaine Active Directory** et **Serveur DNS** sont sélectionnés. La création du domaine infraready.test est ensuite confirmé par la console AD, les zones DNS et les tests Kerberos présentés dans les pages suivantes.

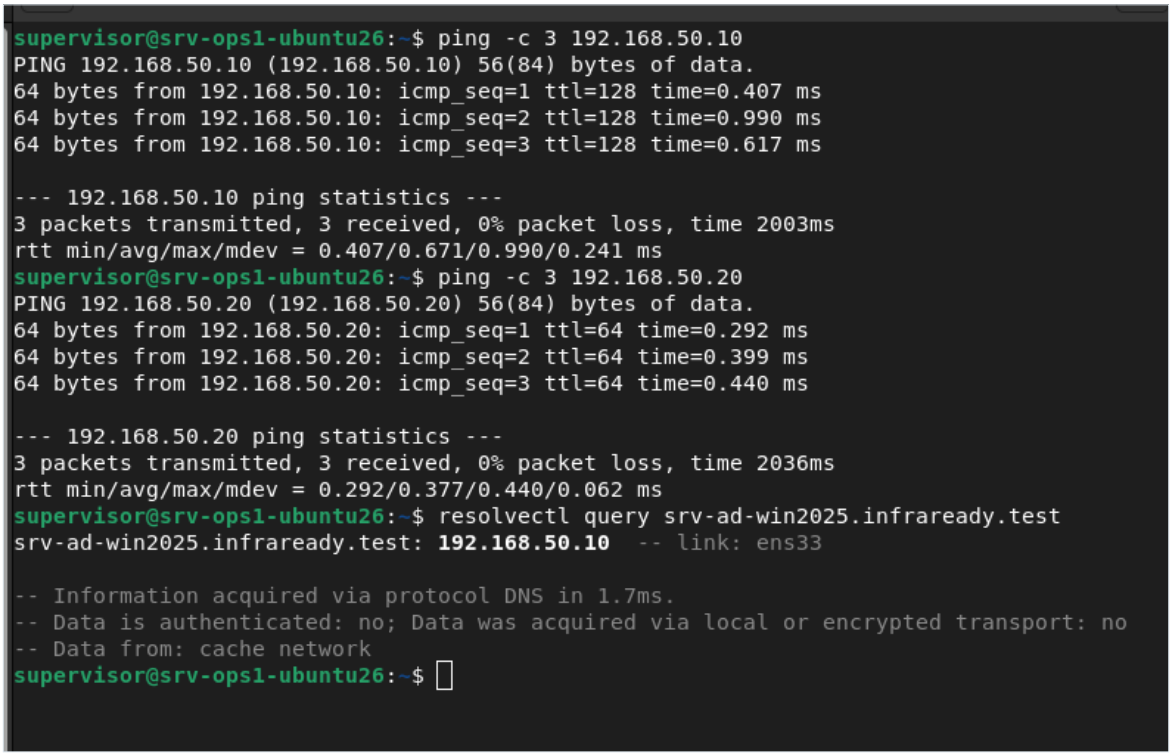


F04. Sélection des rôles Windows AD DS et DNS dans l'assistant. [AD, p. 2]

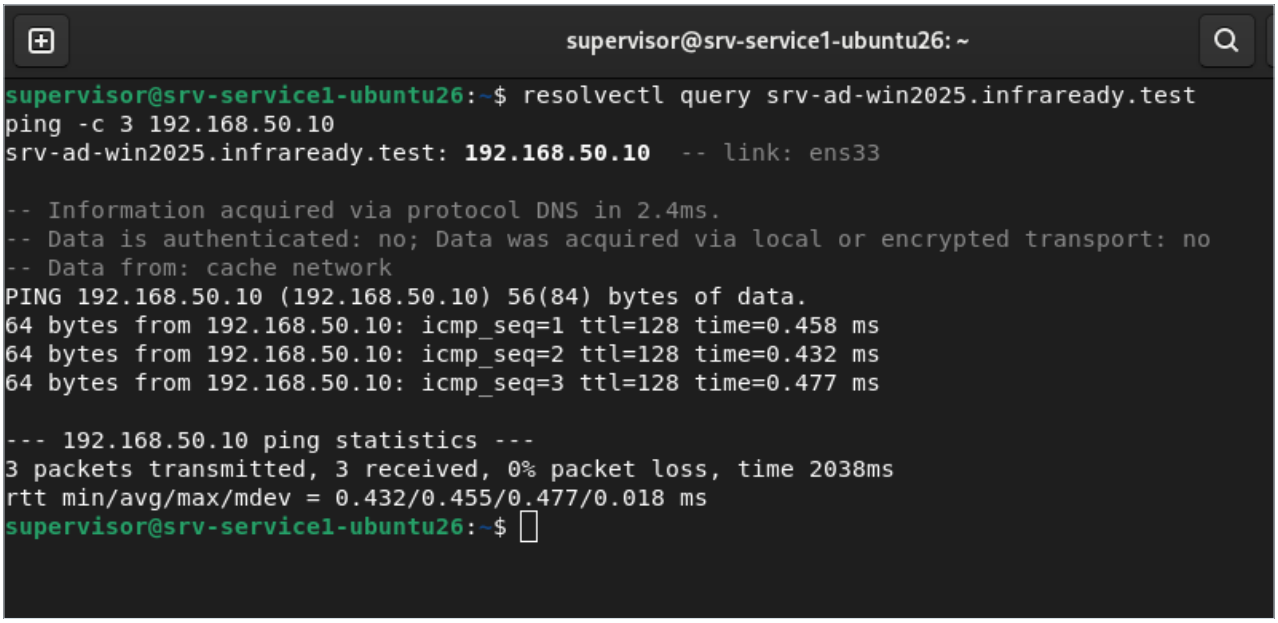
AD DS centralise les identités et groupes. Le DNS permet aux clients de localiser le contrôleur de domaine et les services du domaine, notamment à l'aide des enregistrements SRV.

06. Connectivité entre les serveurs

Les vérifications suivent deux niveaux : ICMP confirme la joignabilité IP, puis une requête DNS vérifie la correspondance entre le nom du contrôleur et son adresse. Un ping réussi ne valide pas à lui seul le service applicatif.



F05. Depuis OPS1 : trois réponses sur trois de AD et SERVICE1, puis résolution du FQDN AD vers 192.168.50.10. [OPS1, p. 1]

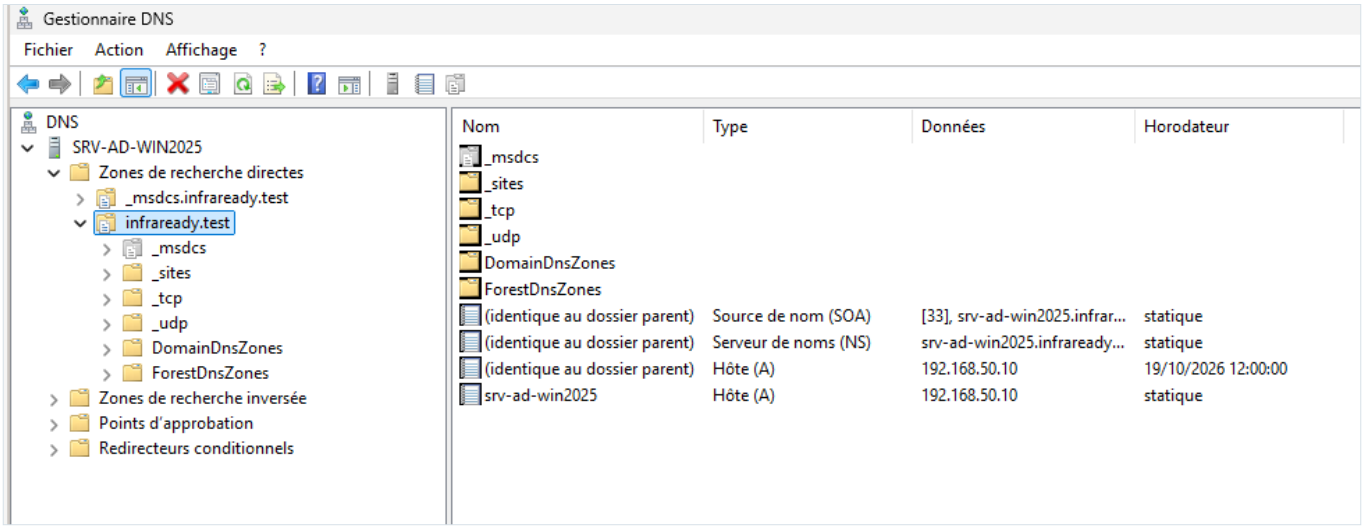


F06. Depuis SERVICE1 : résolution du contrôleur et ping vers AD. [SERVICE, p. 1]

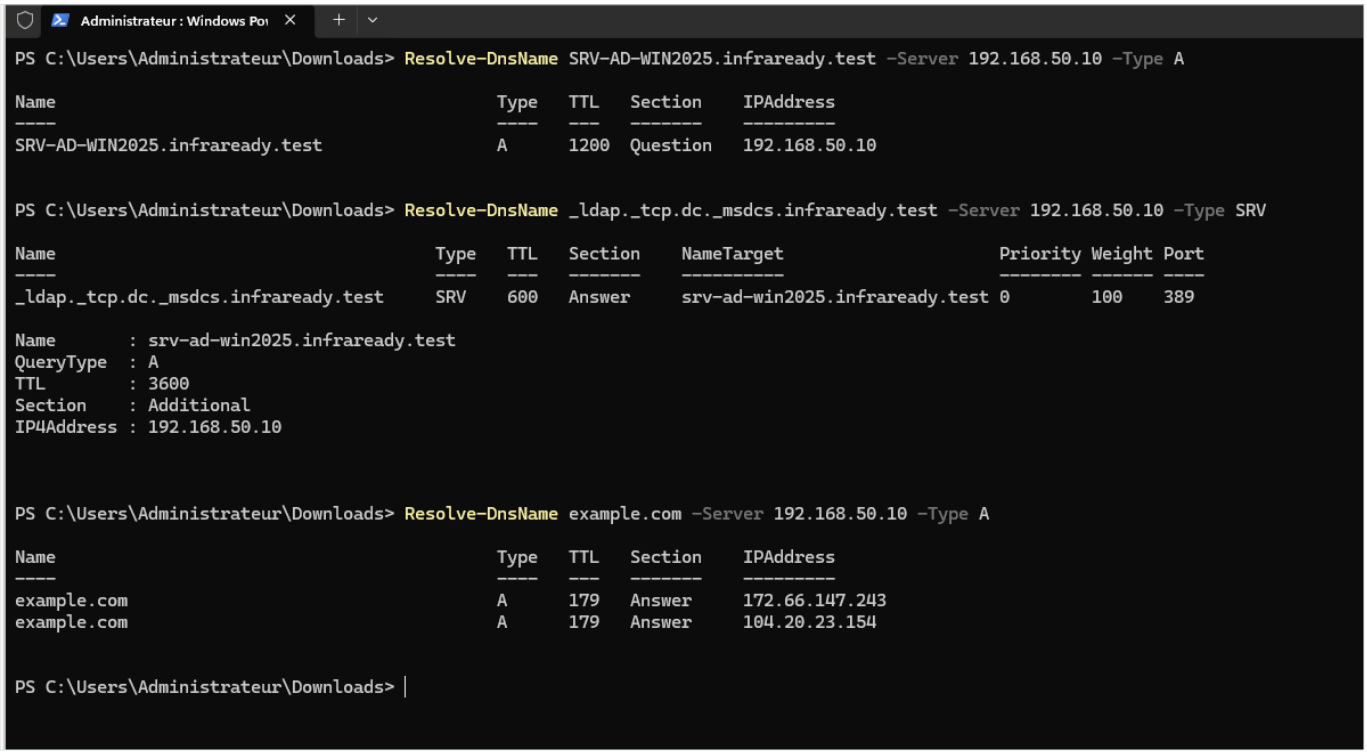
> **Résultat**
> Les chemins OPS1 vers AD, OPS1 vers SERVICE1 et SERVICE1 vers AD sont vérifiés. Les flux applicatifs SSH, WinRM, HTTP et métriques sont testés séparément dans la suite du dossier.

07. DNS du domaine et découverte AD

La zone **infraready.test** contient l'enregistrement A du contrôleur et les sous-domaines de découverte AD. Les tests portent sur l'adresse du serveur, la découverte LDAP et une résolution externe via le DNS du laboratoire.



F07. Zone DNS infraready.test : enregistrements du contrôleur et sous-domaines AD. [AD, p. 4]

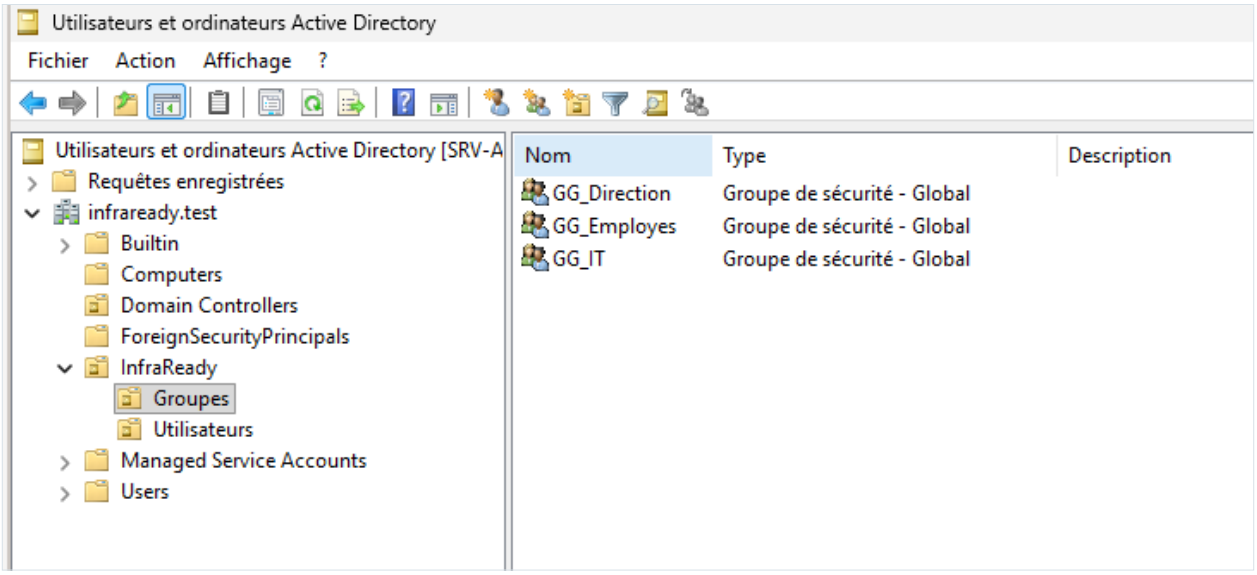


F08. Resolve-DnsName : FQDN AD vers 192.168.50.10 ; SRV LDAP vers contrôleur, port 389 ; réponse pour example.com. [AD, p. 4]

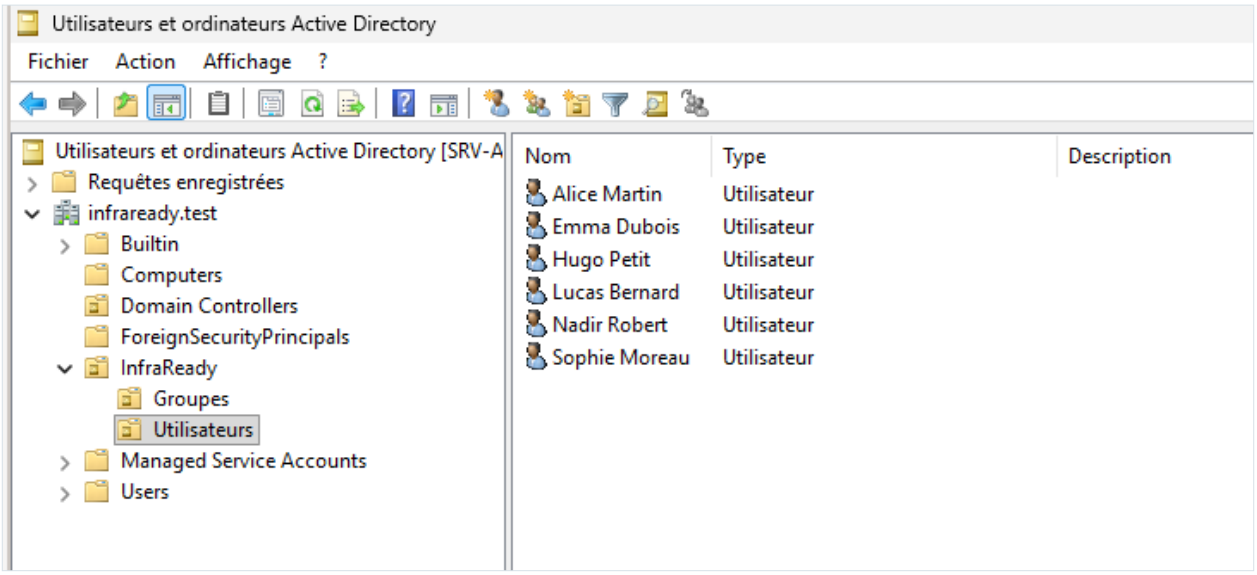
Contrôle	Ce qu'il démontre
Enregistrement A du contrôleur	Le nom srv-ad-win2025.infraready.test est résolu vers la bonne adresse.
_ldap._tcp.dc._msdcs.infraready.test	Le service d'annuaire est annoncé par DNS sur le port LDAP 389.
Requête externe via 192.168.50.10	Le DNS répond aussi pour un domaine externe

08. Comptes et groupes Active Directory

Une unité d'organisation **InfraReady** regroupe deux conteneurs dédiés aux groupes et aux utilisateurs. Cette organisation sépare les objets du projet des conteneurs par défaut et facilite l'administration des droits par groupe.



F09. Trois groupes globaux de sécurité : GG_Direction, GG_Employes et GG_IT. [AD, p. 3]



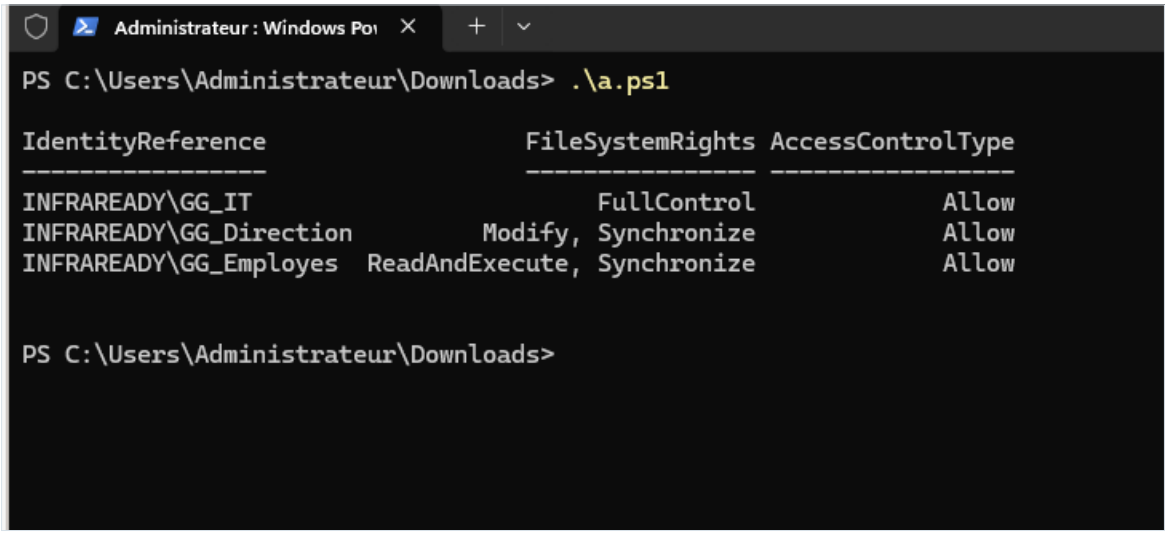
F10. Six comptes de test. [AD, p. 3]

Principe d'attribution des droits

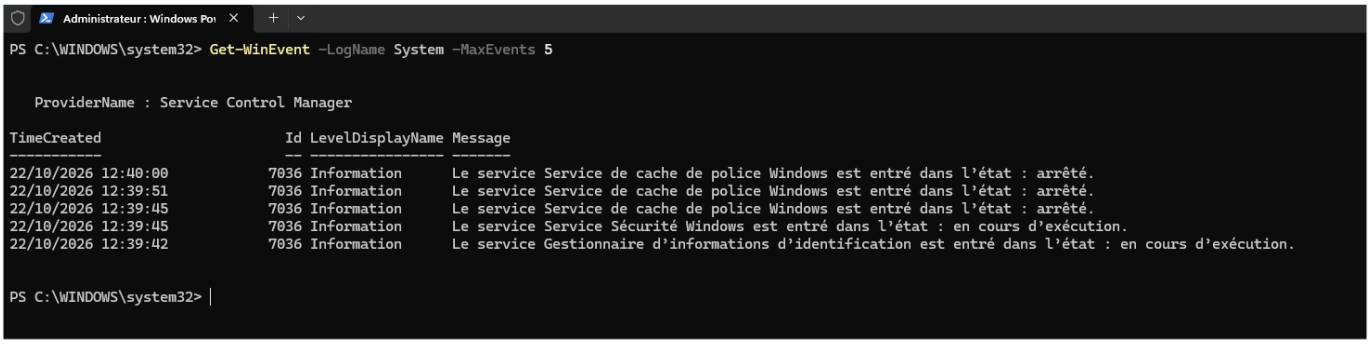
Les autorisations sont associées aux groupes afin d'éviter la multiplication des droits individuels. Une personne reçoit les accès correspondant à son rôle par son appartenance au groupe.

09. Droits NTFS et journaux Windows

Groupe	Autorisation NTFS visible	Portée fonctionnelle
INFRAREADY\GG_IT	FullControl	Administrer le contenu et ses autorisations.
INFRAREADY\GG_Direction	Modify, Synchronize	Lire, créer, modifier et supprimer du contenu.
INFRAREADY\GG_Employes	ReadAndExecute, Synchronize	Lire et exécuter, sans droit de modification explicite.



F11. Liste des autorisations appliquées aux trois groupes. [AD, p. 3]



F12. Get-WinEvent -LogName System -MaxEvents 5 : événements Service Control Manager 7036. [AD, p. 4]

Les événements 7036 consignent les changements d'état de services. Leur date, leur fournisseur et leur message permettent de rapprocher une action d'administration d'un arrêt ou d'un redémarrage. Le journal System complète les contrôles immédiats réalisés avec Get-Service.

10. Service web Linux : Nginx

Nginx fournit un service applicatif simple sur SERVICE1. Son installation et son activation sont automatisées par Ansible ; les vérifications ci-dessous montrent le résultat local et le résultat observé depuis OPS1.

```
supervisor@srv-service1-ubuntu26:~$ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Tue, 20 Oct 2026 08:49:28 GMT
Content-Type: text/html
Content-Length: 615
Last-Modified: Tue, 20 Oct 2026 08:42:23 GMT
Connection: keep-alive
ETag: "6ad7296f-267"
Accept-Ranges: bytes
```

F13. Requête locale : Nginx retourne HTTP/1.1 200 OK et une page HTML de 615 octets. [SERVICE, p. 2]

```
supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ curl -I http://192.168.50.20
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Date: Tue, 20 Oct 2026 08:55:28 GMT
Content-Type: text/html
Content-Length: 615
Last-Modified: Tue, 20 Oct 2026 08:42:23 GMT
Connection: keep-alive
ETag: "6ad7296f-267"
Accept-Ranges: bytes

supervisor@srv-ops1-ubuntu26:~/infraready-ansible$
```

F14. Depuis OPS1, http://192.168.50.20 répond également 200 OK. [OPS1, p. 4]

```
supervisor@srv-service1-ubuntu26:~$ ls -ld /var/www/html /var/www/html/*
drwxr-xr-x 2 root root 4096 Oct 20 08:42 /var/www/html
-rw-r--r-- 1 root root  615 Oct 20 08:42 /var/www/html/index.nginx-debian.html
supervisor@srv-service1-ubuntu26:~$
```

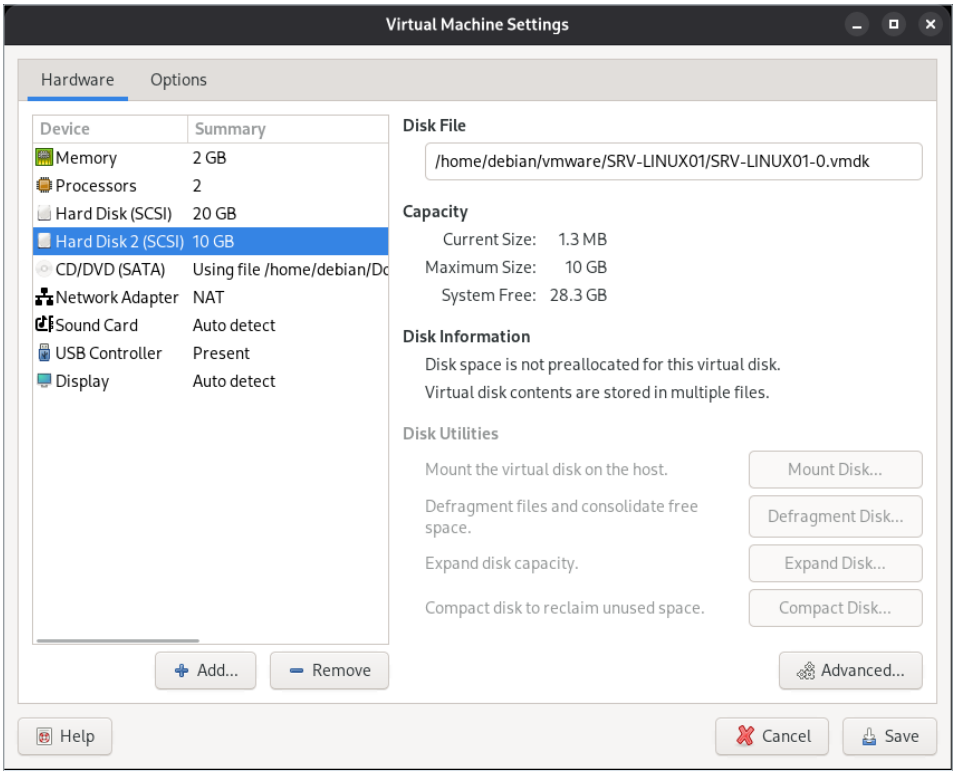
F15. Permissions de /var/www/html et de la page par défaut : propriété root:root. [SERVICE, p. 2]

Lecture des permissions

Le répertoire web est en **755** et le fichier en **644** : root peut modifier le contenu, les autres utilisateurs peuvent le lire et traverser le répertoire. Le serveur web peut donc servir la page sans en être propriétaire ni disposer d'un droit d'écriture sur celle-ci.

11. Stockage : ajout d'un disque de données

Un second disque virtuel de 10 Go est ajouté à SERVICE1 pour isoler les données du système. La capture VMware distingue le disque système de 20 Go du disque de données. Dans Linux, lsblk permet de confirmer la cible avant toute modification.



F16. Configuration VMware : disque SCSI supplémentaire de 10 Go ; réseau NAT. [SERVICE, p. 3]

```
supervisor@srv-service1-ubuntu26:~$ lsblk
NAME                      MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0                       7:0    0  1,5M  1 loop /snap/hwctl/131
loop1                       7:1    0 66,8M  1 loop /snap/core24/1643
loop2                       7:2    0  1,5M  1 loop /snap/hwctl/123
loop3                       7:3    0 50,1M  1 loop /snap/snapd/27710
sda                         8:0    0   20G  0 disk
├─sda1                      8:1    0    1M  0 part
├─sda2                      8:2    0  1,8G  0 part /boot
└─sda3                      8:3    0 18,2G  0 part
  └─ubuntu--vg-ubuntu--lv 252:0    0   10G  0 lvm  /
sdb                         8:16   0   10G  0 disk
sr0                        11:0    1  2,7G  0 rom
supervisor@srv-service1-ubuntu26:~$
```

F17. lsblk identifie /dev/sdb comme un disque de 10 Go encore non partitionné. [SERVICE, p. 3]

> **Précaution avant partitionnement**
> La cible du TP est /dev/sdb. Sur une autre VM, ce nom peut désigner un autre disque : vérifier la taille, les partitions et les montages avec lsblk avant de lancer fdisk ou mkfs. Le disque système /dev/sda ne doit pas être formaté.

12. Partitionnement GPT et système ext4

Le disque de données reçoit une table GPT et une partition utilisant l'espace disponible. Dans fdisk, **g** crée la table GPT, **n** la partition et **w** écrit les changements. Le système de fichiers ext4 est ensuite créé sur la partition /dev/sdb1.

```
supervisor@srv-service1-ubuntu26:~$ sudo fdisk /dev/sdb
[sudo: authenticate] Password:

Welcome to fdisk (util-linux 2.41.3).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS (MBR) disklabel with disk identifier 0xb570d2ee.

Command (m for help): g
Created a new GPT disklabel (GUID: 6862DB1E-2DB2-4632-9A74-47F830EA3A5D).

Command (m for help): n
Partition number (1-128, default 1):
First sector (2048-20971486, default 2048):
Last sector, +/-sectors or +/-size{K,M,G,T,P} (2048-20971486, default 20969471):

Created a new partition 1 of type 'Linux filesystem' and of size 10 GiB.

Command (m for help): w
The partition table has been altered.
Calling ioctl() to re-read partition table.
Syncing disks.

supervisor@srv-service1-ubuntu26:~$ lsblk /dev/sdb
NAME   MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sdb      8:16   0  10G  0 disk
└─sdb1   8:17   0  10G  0 part
supervisor@srv-service1-ubuntu26:~$
```

F18. Création d'une partition GPT de 10 GiB ; lsblk confirme /dev/sdb1. [SERVICE, p. 4]

```
supervisor@srv-service1-ubuntu26:~$ sudo mkfs.ext4 /dev/sdb1
mke2fs 1.47.2 (1-Jan-2025)
Creating filesystem with 2620928 4k blocks and 655360 inodes
Filesystem UUID: 913a8c3a-e094-4fd5-ba82-99b2bff500ec
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

supervisor@srv-service1-ubuntu26:~$ lsblk -f /dev/sdb
NAME   FSTYPE FSVER LABEL UUID                                 FSAVAIL FSUSE% MOUNTPOINTS
sdb
└─sdb1 ext4   1.0         913a8c3a-e094-4fd5-ba82-99b2bff500ec
supervisor@srv-service1-ubuntu26:~$
```

F19. mkfs.ext4 puis lsblk -f : système de fichiers ext4 et UUID attribué. [SERVICE, p. 4]

L'UUID identifie le système de fichiers indépendamment du nom /dev/sdb1. Il est utilisé dans /etc/fstab pour fiabiliser le montage lors des démarrages suivants. Le formatage est une opération initiale destructive, à ne pas rejouer comme une simple vérification.

13. Montage et persistance avec fstab

Le point de montage **/srv/infraready** est créé, puis /dev/sdb1 y est monté. df -hT affiche ext4, environ 9,8 Go de capacité et 1 % d'utilisation. Une copie de /etc/fstab est réalisée avant modification.

```
supervisor@srv-service1-ubuntu26:~$ sudo mkdir -p /srv/infraready
supervisor@srv-service1-ubuntu26:~$ sudo mount /dev/sdb1 /srv/infraready
supervisor@srv-service1-ubuntu26:~$ df -hT /srv/infraready
Filesystem     Type  Size  Used Avail Use% Mounted on
/dev/sdb1      ext4  9,8G  2,1M  9,3G   1% /srv/infraready
supervisor@srv-service1-ubuntu26:~$
```

F20. Création du répertoire, montage manuel et contrôle du système de fichiers. [SERVICE, p. 5]

```
 GNU nano 8.7.1                                  /etc/fstab *
# /etc/fstab: static file system information.
#
# Use 'blkid' to print the universally unique identifier for a
# device; this may be used with UUID= as a more robust way to name devices
# that works even if disks are added and removed. See fstab(5).
#
# <file system> <mount point>   <type>  <options>       <dump>  <pass>
# / was on /dev/ubuntu-vg/ubuntu-lv during curtin installation
/dev/disk/by-id/dm-uuid-LVM-HHCSPoe57tEFHQCyUMHiHrPgm3276rPO56w9VJnQF1rQ0Q5G9rYBi7i5P2QYG6gQ / ext4 def>
# /boot was on /dev/sda2 during curtin installation
/dev/disk/by-uuid/322b99e5-6bb7-4abe-8862-dd32c48cee21 /boot ext4 defaults 0 1
/swap.img       none    swap    sw      0       0
UUID=913a8c3a-e094-4fd5-ba82-99b2bff500ec /srv/infraready ext4 defaults 0 2
```

F21. Entrée fstab par UUID : /srv/infraready, ext4, options defaults, dump 0 et passe fsck 2. [SERVICE, p. 5]

Entrée observée dans /etc/fstab (sur une seule ligne)

```
UUID=913a8c3a-e094-4fd5-ba82-99b2bff500ec /srv/infraready ext4 defaults 0 2
```

La valeur 2 place ce volume de données après le système racine dans l'ordre de vérification du système de fichiers.

```
supervisor@srv-service1-ubuntu26:~$ sudo cp /etc/fstab /etc/fstab.bak
supervisor@srv-service1-ubuntu26:~$ sudo nano /etc/fstab
supervisor@srv-service1-ubuntu26:~$ sudo findmnt --verify
none
   [W] non-bind mount source /swap.img is a directory or regular file
   [W] your fstab has been modified, but systemd still uses the old version;
       use 'systemctl daemon-reload' to reload

0 parse errors, 0 errors, 2 warnings
supervisor@srv-service1-ubuntu26:~$
```

F22. Première vérification : aucune erreur, mais systemd utilise encore l'ancienne version de fstab. [SERVICE, p. 5]

L'avertissement conduit à recharger la configuration systemd. Le second avertissement concerne /swap.img, il ne désigne pas le disque de données. La vérification après rechargement apparaît page suivante.

14. Contrôles du montage et des services

```
supervisor@srv-service1-ubuntu26:~$ sudo systemctl daemon-reload
sudo findmnt --verify
none
   [W] non-bind mount source /swap.img is a directory or regular file

0 parse errors, 0 errors, 1 warning
supervisor@srv-service1-ubuntu26:~$
```

F23. Après systemctl daemon-reload : 0 erreur de syntaxe, 0 erreur ; un avertissement lié à /swap.img reste visible. [SERVICE, p. 5]

```
supervisor@srv-service1-ubuntu26:~$ df -hT /srv/infraready
Filesystem     Type  Size  Used Avail Use% Mounted on
/dev/sdb1      ext4  9,8G  2,1M  9,3G   1% /srv/infraready
supervisor@srv-service1-ubuntu26:~$
```

F24. Le volume de données reste monté en ext4 sur /srv/infraready. [SERVICE, p. 5]

La configuration du montage persistant est présente et vérifiée. Ce test complète la validation de persistance, sans reformater le disque.

```
supervisor@srv-service1-ubuntu26:~$ sudo journalctl -u nginx -n 15 --no-pager
[sudo: authenticate] Password:
oct. 20 09:24:07 srv-service1-ubuntu26 systemd[1]: Started nginx.service - A high performance web server
 and a reverse proxy server.
-- Boot 329cf1af4a5c430fb9eb870d29bf03bd --
oct. 20 12:03:53 srv-service1-ubuntu26 systemd[1]: Starting nginx.service - A high performance web serve
r and a reverse proxy server...
oct. 20 12:03:53 srv-service1-ubuntu26 systemd[1]: Started nginx.service - A high performance web server
 and a reverse proxy server.
-- Boot 60b07559f0d842c59d289df00465ed1c --
oct. 20 13:51:05 srv-service1-ubuntu26 systemd[1]: Starting nginx.service - A high performance web serve
r and a reverse proxy server...
oct. 20 13:51:05 srv-service1-ubuntu26 systemd[1]: Started nginx.service - A high performance web server
 and a reverse proxy server.
oct. 20 14:16:31 srv-service1-ubuntu26 systemd[1]: Stopping nginx.service - A high performance web serve
r and a reverse proxy server...
oct. 20 14:16:31 srv-service1-ubuntu26 systemd[1]: nginx.service: Deactivated successfully.
oct. 20 14:16:31 srv-service1-ubuntu26 systemd[1]: Stopped nginx.service - A high performance web server
 and a reverse proxy server.
-- Boot 4a8d92cbcff24adb870a34bc99f91751 --
oct. 20 14:17:25 srv-service1-ubuntu26 systemd[1]: Starting nginx.service - A high performance web serve
r and a reverse proxy server...
oct. 20 14:17:25 srv-service1-ubuntu26 systemd[1]: Started nginx.service - A high performance web server
 and a reverse proxy server.
oct. 20 14:27:26 srv-service1-ubuntu26 systemd[1]: Stopping nginx.service - A high performance web serve
r and a reverse proxy server...
oct. 20 14:27:26 srv-service1-ubuntu26 systemd[1]: nginx.service: Deactivated successfully.
oct. 20 14:27:26 srv-service1-ubuntu26 systemd[1]: Stopped nginx.service - A high performance web server
 and a reverse proxy server.
-- Boot 660ce76beae344e38aff9317afc91630 --
oct. 20 14:27:42 srv-service1-ubuntu26 systemd[1]: Starting nginx.service - A high performance web serve
r and a reverse proxy server...
oct. 20 14:27:43 srv-service1-ubuntu26 systemd[1]: Started nginx.service - A high performance web server
 and a reverse proxy server.
supervisor@srv-service1-ubuntu26:~$
```

F25. journalctl -u nginx : démarrages, arrêts propres et changements de session de démarrage. [SERVICE, p. 6]

Les journaux Nginx montrent le cycle de vie du service et le dernier démarrage réussi. Ils sont utiles pour distinguer un arrêt explicite, un redémarrage de VM et une erreur de lancement. Le suivi des processus peut être complété par systemctl status nginx et ps.

15. Comptes et permissions Linux

Le compte de test **ir_test** appartient au groupe **infraready**. Un répertoire partagé est créé sur le volume de données, avec un propriétaire root, un groupe infraready et le mode 2770.

```
supervisor@srv-service1-ubuntu26:~$ id ir_test
getent group infraready
uid=1001(ir_test) gid=1002(ir_test) groups=1002(ir_test),1001(infraready)
infraready:x:1001:ir_test
supervisor@srv-service1-ubuntu26:~$
```

F26. id et getent confirment l'utilisateur ir_test et son appartenance au groupe infraready. [SERVICE, p. 6]

Élément du mode 2770	Effet recherché
2 : bit setgid sur le répertoire	Les nouveaux objets héritent du groupe infraready.
7 pour le propriétaire / 7 pour le groupe	Lecture, écriture et traversée du répertoire autorisées.
0 pour les autres utilisateurs	Aucun accès direct au répertoire pour les utilisateurs hors groupe.

```
supervisor@srv-service1-ubuntu26:~$ sudo mkdir /srv/infraready/partage
sudo chown root:infraready /srv/infraready/partage
sudo chmod 2770 /srv/infraready/partage
supervisor@srv-service1-ubuntu26:~$ ls -ld /srv/infraready/partage
sudo -u ir_test touch /srv/infraready/partage/test.txt
ls -l /srv/infraready/partage
drwxrws--- 2 root infraready 4096 Oct 22 10:45 /srv/infraready/partage
ls: impossible d'ouvrir le répertoire '/srv/infraready/partage' : Permission refusée
supervisor@srv-service1-ubuntu26:~$
```

F27. chown root:infraready et chmod 2770 ; création avec ir_test ; lecture refusée au compte courant supervisor. [SERVICE, p. 6]

```
supervisor@srv-service1-ubuntu26:~$ sudo -u ir_test ls -l /srv/infraready/partage
total 0
-rw-r--r-- 1 ir_test infraready 0 Oct 22 10:45 test.txt
supervisor@srv-service1-ubuntu26:~$
```

F28. Lecture réussie avec ir_test : test.txt appartient à ir_test:infraready. [SERVICE, p. 7]

> **Interprétation du test**
> Le refus affiché concerne la commande ls lancée sous supervisor. Le test suivant sous ir_test confirme l'accès autorisé. Le fichier créé est en 644 : setgid transmet le groupe, mais ne rend pas automatiquement le fichier modifiable par tous les membres du groupe.

16. Ansible : prérequis et accès SSH

OPS1 est le contrôleur Ansible. SERVICE1 est administré avec SSH ; le serveur Windows utilise WinRM et Kerberos. Les dépendances de contrôle doivent être installées dans l'environnement Python employé par Ansible, et les noms du domaine doivent être résolus avant les tests Windows.

Composant	Préparation / version documentée
Contrôleur OPS1	Ansible, outils Kerberos, pywinrm avec support Kerberos ; collection ansible.windows 3.8.0 dans requirements.yml.
SERVICE1	SSH, compte supervisor, clé publique autorisée, Python /usr/bin/python3 ; sudo.ws référencé pour l'élévation.
AD Windows	WinRM TCP 5985, compte de domaine et ticket Kerberos valide. Le dépôt impose le chiffrement des messages.

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_infraready -C "infraready-ops1"
Generating public/private ed25519 key pair.
Enter passphrase for "/home/supervisor/.ssh/id_ed25519_infraready" (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/supervisor/.ssh/id_ed25519_infraready
Your public key has been saved in /home/supervisor/.ssh/id_ed25519_infraready.pub
The key fingerprint is:
SHA256:QMm9m97EKFLQErBiTPSHQyt1S364oh0SxD3nu3W5NQM infraready-ops1
The key's randomart image is:
+--[ED25519 256]--+
|o+o+.*.o         |
|+.++Oo* .        |
|.=.=+B.. .       |
|..o o.+..E       |
| . o o. S=o      |
|  + +...+oo+     |
| . . .oo.oo o    |
|     .  ...      |
|                 |
+----[SHA256]-----+
supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ ssh-copy-id -i ~/.ssh/id_ed25519_infraready.pub super
visor@192.168.50.20
/usr/bin/ssh-copy-id: INFO: Source of key(s) to be installed: "/home/supervisor/.ssh/id_ed25519_infrarea
dy.pub"
The authenticity of host '192.168.50.20 (192.168.50.20)' can't be established.
ED25519 key fingerprint is: SHA256:LRFTEQnx+MzZXWThi5a5XRxhBY/VgCtA4TXuPRC90Lo
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
/usr/bin/ssh-copy-id: INFO: attempting to log in with the new key(s), to filter out any that are already
 installed
/usr/bin/ssh-copy-id: INFO: 1 key(s) remain to be installed -- if you are prompted now it is to install
the new keys
supervisor@192.168.50.20's password:

Number of key(s) added: 1

Now try logging into the machine, with: "ssh -i /home/supervisor/.ssh/id_ed25519_infraready 'supervisor@
192.168.50.20'"
and check to make sure that only the key(s) you wanted were added.
```

F29. Génération d'une clé Ed25519 sur OPS1, puis copie de la clé publique sur SERVICE1 ; une clé ajoutée. [OPS1, p. 2]

La clé privée reste dans /home/supervisor/.ssh/id_ed25519_infraready sur OPS1. Le dépôt ne contient que ce chemin. L'empreinte du serveur SSH doit être vérifiée avant d'accepter la première connexion. Les mots de passe sudo sont demandés avec --ask-become-pass.

17. Inventaire et tests de connexion

L'inventaire sépare les groupes **linux** et **windows**. Les premières captures utilisent le fichier local inventory.ini. Dans le dépôt final, son équivalent est **inventory/hosts.ini** : les commandes de reproduction doivent employer ce chemin.

```
supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ ansible-inventory -i inventory.ini --graph
@all:
  |--@ungrouped:
  |--@linux:
  |  |--srv-service1-ubuntu26
  |--@windows:
  |  |--srv-ad-win2025
supervisor@srv-ops1-ubuntu26:~/infraready-ansible$
```

F30. Graphe d'inventaire : SERVICE1 dans linux et AD dans windows. [OPS1, p. 3]

```
supervisor@srv-ops1-ubuntu26:~/infraready-ansible$
cat > inventory.ini <<'EOF'
[linux]
srv-service1-ubuntu26 ansible_host=192.168.50.20 ansible_user=supervisor ansible_ssh_private_key_file=/h
ome/supervisor/.ssh/id_ed25519_infraready
EOF
supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ ansible -i inventory.ini linux -m ansible.builtin.pin
g
[WARNING]: Host 'srv-service1-ubuntu26' is using the discovered Python interpreter at '/usr/bin/python3.
14', but future installation of another Python interpreter could cause a different interpreter to be dis
covered. See https://docs.ansible.com/ansible-core/2.20/reference_appendices/interpreter_discovery.html
for more information.
srv-service1-ubuntu26 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.14"
    },
    "changed": false,
    "ping": "pong"
}
supervisor@srv-ops1-ubuntu26:~/infraready-ansible$
```

F31. Test ansible.builtin.ping vers SERVICE1 : SUCCESS, pong, changed=false. [OPS1, p. 2]

```
(infraready) supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ ansible -i inventory.ini windows -m ansi
ble.windows.win_ping
srv-ad-win2025 | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
(infraready) supervisor@srv-ops1-ubuntu26:~/infraready-ansible$
```

F32. Test ansible.windows.win_ping vers Windows : SUCCESS et pong. [OPS1, p. 5]

Ces modules vérifient une session d'administration et l'exécution du module distant, ce n'est pas des pings ICMP. L'avertissement initial sur la découverte de Python n'est pas bloquant. Le dépôt final fixe ansible_python_interpreter à /usr/bin/python3.

```
ansible-inventory -i inventory/hosts.ini --graph
ansible -i inventory/hosts.ini linux -m ansible.builtin.ping
ansible -i inventory/hosts.ini windows -m ansible.windows.win_ping
```

18. Playbook Linux : installation de Nginx

Le playbook cible le groupe linux avec élévation de privilèges. Le module apt installe Nginx et actualise le cache si nécessaire, le module service impose un service démarré et activé au démarrage. Le cache est considéré valide pendant 3 600 secondes.

```
 GNU nano 8.7.1                          playbooks/nginx.yml *

---
- name: Installer et configurer Nginx sur SERVICE1
  hosts: linux
  become: true

  tasks:
    - name: Installer Nginx
      ansible.builtin.apt:
        name: nginx
        state: present
        update_cache: true
        cache_valid_time: 3600

    - name: Demarrer et activer Nginx
      ansible.builtin.service:
        name: nginx
        state: started
        enabled: true
```

F33. Playbook nginx.yml : installation du paquet puis activation du service. [OPS1, p. 3]

```
supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ ansible-playbook -i inventory.ini playbooks/nginx.yml
 --ask-become-pass
BECOME password:

PLAY [Installer et configurer Nginx sur SERVICE1] ******************************************************

TASK [Gathering Facts] *********************************************************************************
ok: [srv-service1-ubuntu26]

TASK [Installer Nginx] *********************************************************************************
changed: [srv-service1-ubuntu26]

TASK [Demarrer et activer Nginx] ***********************************************************************
ok: [srv-service1-ubuntu26]

PLAY RECAP *********************************************************************************************
srv-service1-ubuntu26      : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0
 ignored=0

supervisor@srv-ops1-ubuntu26:~/infraready-ansible$
```

F34. Première exécution : ok=3, changed=1, unreachable=0 et failed=0. [OPS1, p. 3]

Commande depuis la racine du dépôt

```
ansible-playbook -i inventory/hosts.ini \
  playbooks/nginx.yml --ask-become-pass
```

Le changement correspond à l'installation du paquet. Le service est ensuite dans l'état attendu. Le contrôle HTTP local et distant (page 12) complète le récapitulatif Ansible avec une vérification fonctionnelle. Les tâches utilisent des modules d'état plutôt que des commandes shell répétées. [R2]

19. Idempotence et authentification Kerberos

Un second lancement du playbook Nginx conserve la configuration : **ok=3, changed=0, failed=0**. Ce résultat démontre l'idempotence sur l'état observé. Une modification ultérieure des paquets, du cache ou du service peut légitimement produire un nouveau changement.

```
supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ ansible-playbook -i inventory.ini playbooks/nginx.yml
 --ask-become-pass
BECOME password:

PLAY [Installer et configurer Nginx sur SERVICE1] ******************************************************

TASK [Gathering Facts] *********************************************************************************
ok: [srv-service1-ubuntu26]

TASK [Installer Nginx] *********************************************************************************
ok: [srv-service1-ubuntu26]

TASK [Demarrer et activer Nginx] ***********************************************************************
ok: [srv-service1-ubuntu26]

PLAY RECAP *********************************************************************************************
srv-service1-ubuntu26      : ok=3    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
 ignored=0

supervisor@srv-ops1-ubuntu26:~/infraready-ansible$
```

F35. Deuxième exécution Linux sans modification. [OPS1, p. 4]

Préparer la connexion Windows

La résolution de l'enregistrement _kerberos._tcp.infraready.test localise le contrôleur sur le port 88. kinit demande un ticket pour le principal de domaine, klist confirme la présence du ticket TGT et sa période de validité.

```
supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ resolvectl query -t SRV _kerberos._tcp.infraready.tes
t
_kerberos._tcp.INFRAREADY.TEST IN SRV 0 100 88 srv-ad-win2025.infraready.test -- link: ens33

-- Information acquired via protocol DNS in 666us.
-- Data is authenticated: no; Data was acquired via local or encrypted transport: no
-- Data from: cache
```

F36. Découverte du service Kerberos par DNS SRV. [OPS1, p. 4]

```
supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ kinit Administrateur@INFRAREADY.TEST
Password for Administrateur@INFRAREADY.TEST:
supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: Administrateur@INFRAREADY.TEST

Valid starting       Expires              Service principal
20/10/2026 09:49:35  20/10/2026 19:49:35  krbtgt/INFRAREADY.TEST@INFRAREADY.TEST
        renew until 21/10/2026 09:49:25
supervisor@srv-ops1-ubuntu26:~/infraready-ansible$
```

F37. Obtention d'un TGT Administrateur@INFRAREADY.TEST et vérification avec klist. [OPS1, p. 4]

> **Configuration versionnée**
> WinRM utilise le FQDN du contrôleur, le port 5985, kerberos, kinit_mode: manual et message_encryption: always. La délégation Kerberos est désactivée. Le ticket doit être valide lors de l'exécution, DNS et synchronisation horaire sont des prérequis. [R1]

20. Playbook Windows : état et vérifications

Le playbook Windows réalise cinq opérations : vérifier la connexion, créer un répertoire, y déposer un fichier, lire l'état du service DNS et échouer explicitement si ce service est absent ou arrêté.

```
  GNU nano 8.7.1          /home/supervisor/infraready-ansible/playbooks/windows.yml
---
- name: Configurer et verifier le serveur Windows InfraReady
  hosts: windows
  gather_facts: false

  tasks:
    - name: Verifier la connexion Windows
      ansible.windows.win_ping:

    - name: Creer le repertoire d'automatisation InfraReady
      ansible.windows.win_file:
        path: 'C:\InfraReady\Automatisation'
        state: directory

    - name: Deployer un fichier de configuration de test
      ansible.windows.win_copy:
        dest: 'C:\InfraReady\Automatisation\infraready.txt'
        content: |
          Projet : InfraReady
          Serveur : SRV-AD-WIN2025
          Configuration geree par Ansible depuis OPS1

    - name: Verifier le service DNS Windows
      ansible.windows.win_service_info:
        name: DNS
      register: dns_info

    - name: Valider que le service DNS fonctionne
      ansible.builtin.assert:
        that:
          - dns_info.exists
          - dns_info.services[0].state == 'started'
        fail_msg: "Le service DNS Windows est absent ou arrete."
        success_msg: "Le service DNS Windows est actif."
```

F38. Playbook windows.yml : win_ping, win_file, win_copy, win_service_info puis assert. [OPS1, p. 5]

Opération	Résultat attendu
win_file / win_copy	C:\InfraReady\Automatisation et son fichier infraready.txt existent avec le contenu prévu.
win_service_info + assert	dns_info.exists est vrai et le service DNS est dans l'état started.
gather_facts: false	Seules les informations nécessaires au playbook sont demandées.

La vérification intégrée porte sur l'état du service DNS. Le fonctionnement de la résolution lui-même est contrôlé par les requêtes présentées page 9. Ce playbook ne déploie pas le domaine AD ni les mises à jour Windows.

21. Résultats Windows et compte de test

```
(infraready) supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ ansible-playbook -i inventory.ini playbo
oks/windows.yml

PLAY [Configurer et verifier le serveur Windows InfraReady] ********************************************

TASK [Verifier la connexion Windows] *******************************************************************
ok: [srv-ad-win2025]

TASK [Creer le repertoire d'automatisation InfraReady] *************************************************
ok: [srv-ad-win2025]

TASK [Deployer un fichier de configuration de test] ****************************************************
ok: [srv-ad-win2025]

TASK [Verifier le service DNS Windows] *****************************************************************
ok: [srv-ad-win2025]

TASK [Valider que le service DNS fonctionne] ***********************************************************
ok: [srv-ad-win2025] => {
    "changed": false,
    "msg": "Le service DNS Windows est actif."
}

PLAY RECAP *********************************************************************************************
srv-ad-win2025             : ok=5    changed=0    unreachable=0    failed=0    skipped=0    rescued=0
 ignored=0
```

F39. Exécution Windows : cinq tâches réussies, changed=0, unreachable=0, failed=0 ; assertion DNS positive. [OPS1, p. 6]

Le récapitulatif confirme l'exécution complète ainsi que le bon fonctionnement du service DNS.

Authentification d'un utilisateur AD depuis Linux

```
supervisor@srv-ops1-ubuntu26:~$ kinit alice.martin@INFRAREADY.TEST
Password for alice.martin@INFRAREADY.TEST:
Password expired.  You must change it now.
Enter new password:
Enter it again:
supervisor@srv-ops1-ubuntu26:~$ klist
Ticket cache: FILE:/tmp/krb5cc_1000
Default principal: alice.martin@INFRAREADY.TEST

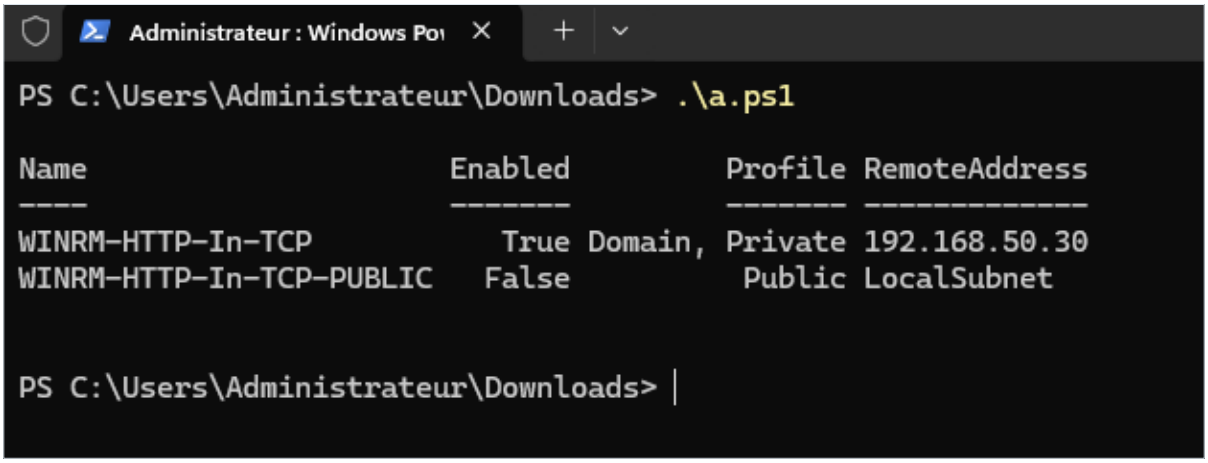
Valid starting       Expires              Service principal
22/10/2026 10:51:12  22/10/2026 20:51:12  krbtgt/INFRAREADY.TEST@INFRAREADY.TEST
        renew until 23/10/2026 10:51:12
supervisor@srv-ops1-ubuntu26:~$
```

F40. Le mot de passe expiré d'alice.martin est renouvelé, puis klist affiche un TGT valide. [OPS1, p. 12]

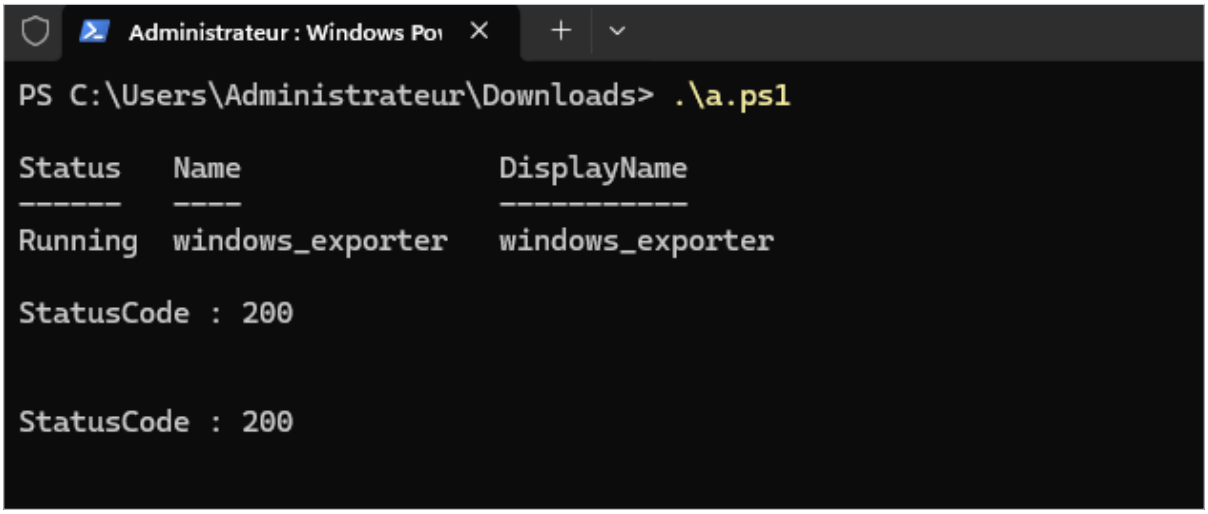
Le test démontre qu'OPS1 peut obtenir un ticket auprès du domaine pour un compte de test, après renouvellement du mot de passe demandé par Kerberos.

22. WinRM et Windows Exporter

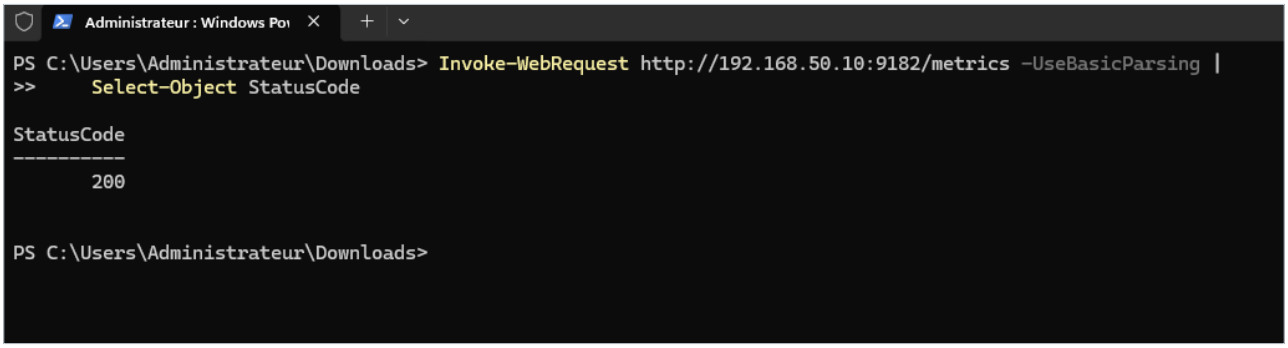
Le pare-feu Windows restreint la règle WinRM HTTP aux connexions provenant d'OPS1. Windows Exporter fournit les métriques sur TCP 9182, son service et son endpoint HTTP sont vérifiés séparément.



F41. Règle WinRM active sur Domain/Private et limitée à 192.168.50.30 ; règle Public désactivée. [AD, p. 5]



F42. windows_exporter est Running ; les contrôles HTTP retournent le code 200. [AD, p. 5]



F43. Test local de http://192.168.50.10:9182/metrics avec Invoke-WebRequest. [AD, p. 5]

23. Procédure de maintenance contrôlée

La maintenance doit conserver une possibilité de retour arrière et permettre de comparer l'état des services avant et après intervention. Les mises à jour Linux sont automatisées, celles de Windows utilisent l'interface native Windows Update, comme l'autorise le sujet.

Étape	Procédure	Critère de poursuite
1. Préparer	Identifier la VM et l'impact, noter les versions et l'état des services. Vérifier l'espace libre avec df -h ou les outils Windows.	Cible confirmée et espace suffisant.
2. Protéger	Créer un snapshot VMware avant le changement majeur, conserver la console disponible.	Point de retour identifié. La capture montre la préparation du snapshot Windows.
3. Mettre à jour	Lancer maintenance.yml sur SERVICE1, installer les correctifs proposés par Windows Update sur AD.	Aucune erreur bloquante, résultats conservés.
4. Redémarrer si requis	Planifier l'interruption, vérifier les dépendances, notamment DNS/AD et les collecteurs, avant le redémarrage.	Redémarrage contrôlé, sans couper les VM arbitrairement.
5. Contrôler	Vérifier services, HTTP, DNS, métriques et éventuel redémarrage restant.	État attendu retrouvé et tests positifs.
6. Clore	Documenter les opérations et anomalies. Conserver les preuves et décider du maintien temporaire du snapshot.	Maintenance tracée, action corrective si un test échoue.

Contrôles Linux avant et après

```
df -h
systemctl is-active nginx prometheus-node-exporter
test -f /run/reboot-required && echo "Redemarrage requis" \
  || echo "Aucun redemarrage requis"
curl -I http://192.168.50.20
```

Contrôles Windows avant et après

```
Get-Service NTDS,DNS
Resolve-DnsName srv-ad-win2025.infraready.test -Server 192.168.50.10
```

En cas de régression, conserver les journaux, identifier le changement concerné et appliquer un retour arrière adapté. Un snapshot facilite une reprise du laboratoire, il ne remplace pas une sauvegarde indépendante, en particulier pour un contrôleur de domaine.

24. Maintenance Linux automatisée

Le contrôle initial montre Nginx et Node Exporter actifs, avec de l'espace disponible sur / et /srv/infraready. Le playbook actualise le cache APT puis demande une mise à niveau safe. Il ne contient pas de redémarrage automatique. [R2]

```
supervisor@srv-service1-ubuntu26:~$ df -h
Filesystem                         Size  Used Avail Use% Mounted on
tmpfs                              306M  1,3M  305M   1% /run
/dev/mapper/ubuntu--vg-ubuntu--lv  9,8G  4,9G  4,4G  53% /
tmpfs                              765M     0  765M   0% /dev/shm
none                               1,0M     0  1,0M   0% /run/credentials/systemd-journald.service
none                               1,0M     0  1,0M   0% /run/credentials/systemd-resolved.service
tmpfs                              765M     0  765M   0% /tmp
/dev/sda2                          1,8G  100M  1,6G   7% /boot
/dev/sdb1                          9,8G  2,1M  9,3G   1% /srv/infraready
none                               1,0M     0  1,0M   0% /run/credentials/systemd-networkd.service
none                               1,0M     0  1,0M   0% /run/credentials/getty@tty1.service
tmpfs                              153M  8,0K  153M   1% /run/user/1000
supervisor@srv-service1-ubuntu26:~$ systemctl is-active nginx prometheus-node-exporter
active
active
supervisor@srv-service1-ubuntu26:~$
```

F44. Avant maintenance : disque racine utilisé à 53 %, données à 1 %, deux services actifs. [PREUVES, p. 1]

```
  GNU nano 8.7.1                                playbo
---
- name: Maintenance Linux InfraReady
  hosts: linux
  become: true

  tasks:
    - name: Mettre a jour les paquets Ubuntu
      ansible.builtin.apt:
        update_cache: true
        upgrade: safe
```

F45. maintenance.yml : apt avec update_cache: true et upgrade: safe. [PREUVES, p. 1]

```
supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ ansible-playbook -i inventory.ini playbooks/maintenan
ce.yml --ask-become-pass
BECOME password:

PLAY [Maintenance Linux InfraReady] ********************************************************************

TASK [Gathering Facts] *********************************************************************************
ok: [srv-service1-ubuntu26]

TASK [Mettre a jour les paquets Ubuntu] ****************************************************************
changed: [srv-service1-ubuntu26]

PLAY RECAP *********************************************************************************************
srv-service1-ubuntu26      : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0
 ignored=0

supervisor@srv-ops1-ubuntu26:~/infraready-ansible$
```

F46. Exécution depuis OPS1 : ok=2, changed=1, unreachable=0, failed=0. [PREUVES, p. 2]

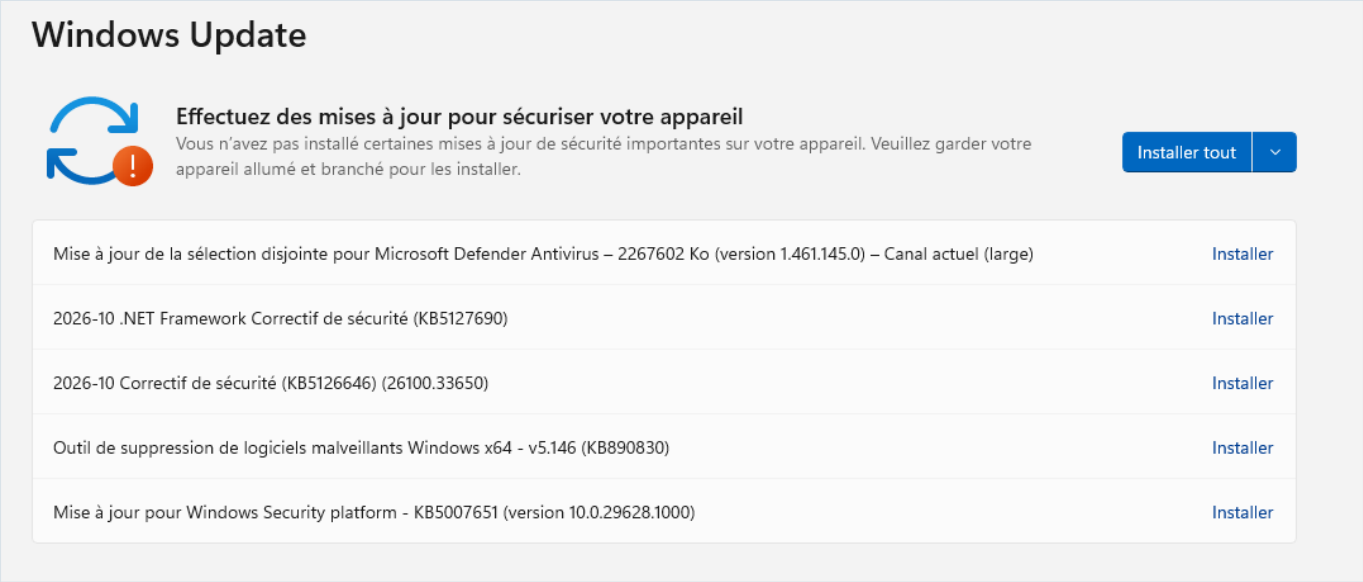
```
supervisor@srv-service1-ubuntu26:~$ systemctl is-active nginx prometheus-node-exporter
active
active
supervisor@srv-service1-ubuntu26:~$ test -f /run/reboot-required && echo "Redemarrage requis" || echo "A
ucun redemarrage requis"
Aucun redemarrage requis
supervisor@srv-service1-ubuntu26:~$
```

F47. Après maintenance : deux services actifs et absence de /run/reboot-required. [PREUVES, p. 2]

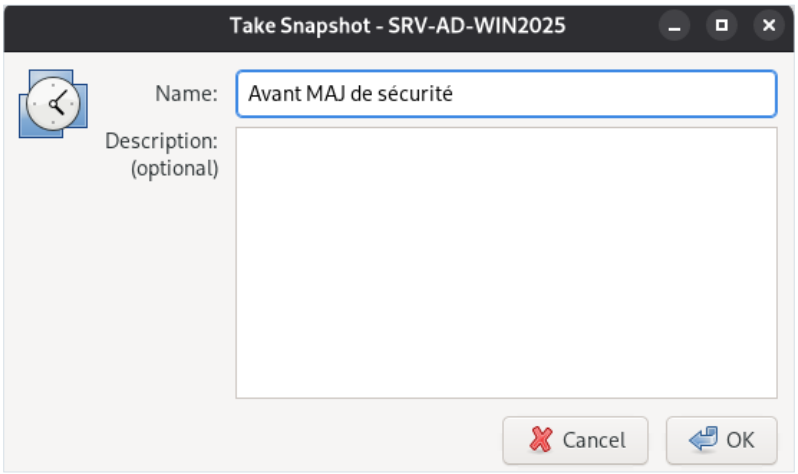
Le contrôle après exécution répond aux critères de maintien des services et de vérification du besoin de redémarrage.

25. Maintenance Windows : préparation

Windows Update présente des mises à jour de sécurité et de composants. Avant leur installation, les services NTDS et DNS sont vérifiés et un snapshot VMware est préparé sous un nom explicite.



F48. Correctifs proposés avant intervention : sécurité Windows, .NET et composants de protection. [PREUVES, p. 2]



F49. Préparation du snapshot « Avant MAJ de sécurité » pour SRV-AD-WIN2025. [PREUVES, p. 3]

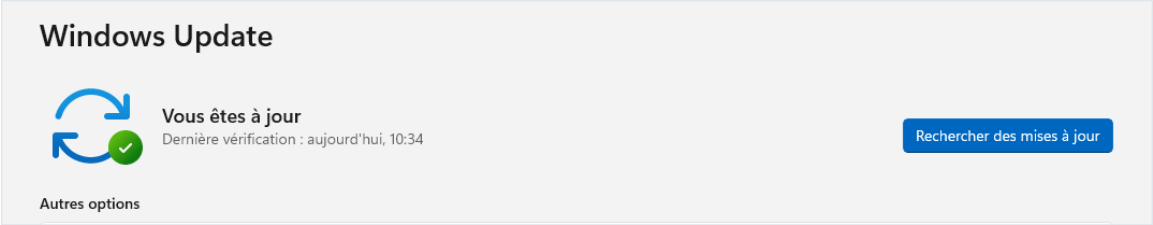
```
PS C:\WINDOWS\system32> Get-Service NTDS,DNS

Status   Name               DisplayName
------   ----               -----------
Running  DNS                Serveur DNS
Running  NTDS               Services de domaine Active Directory

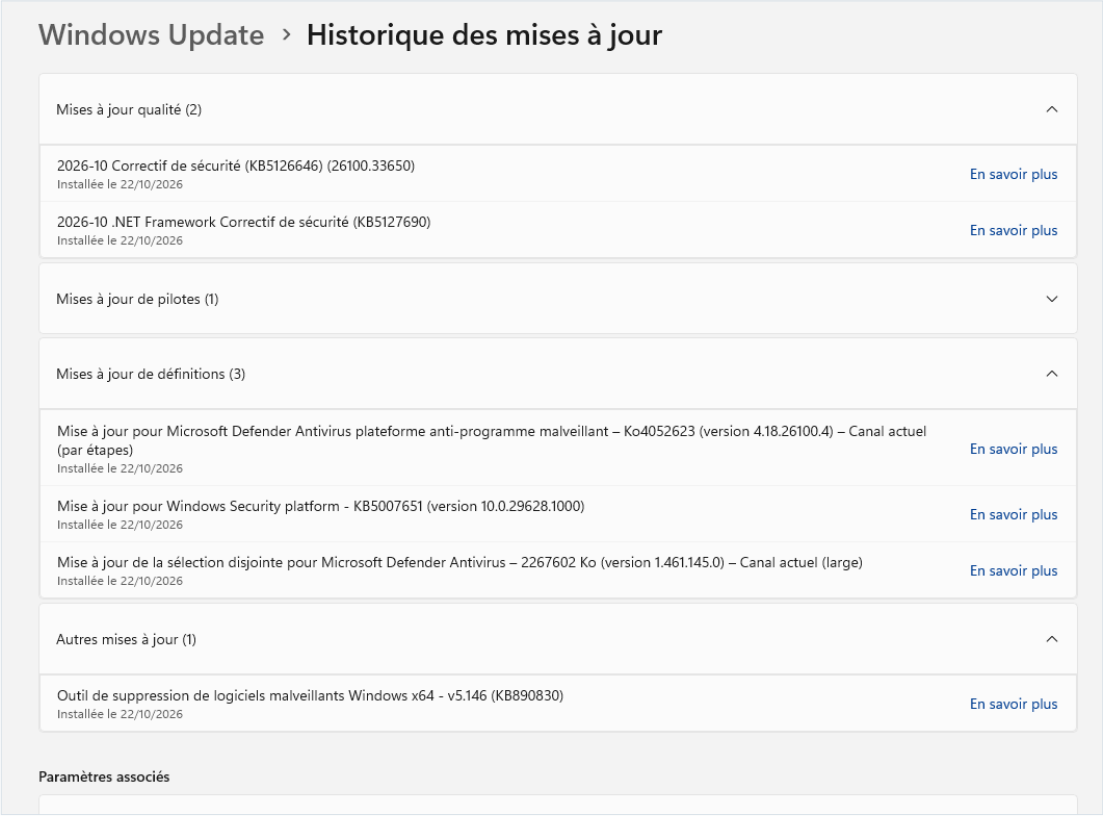

PS C:\WINDOWS\system32>
```

F50. Avant mise à jour : NTDS et DNS sont Running. [PREUVES, p. 3]

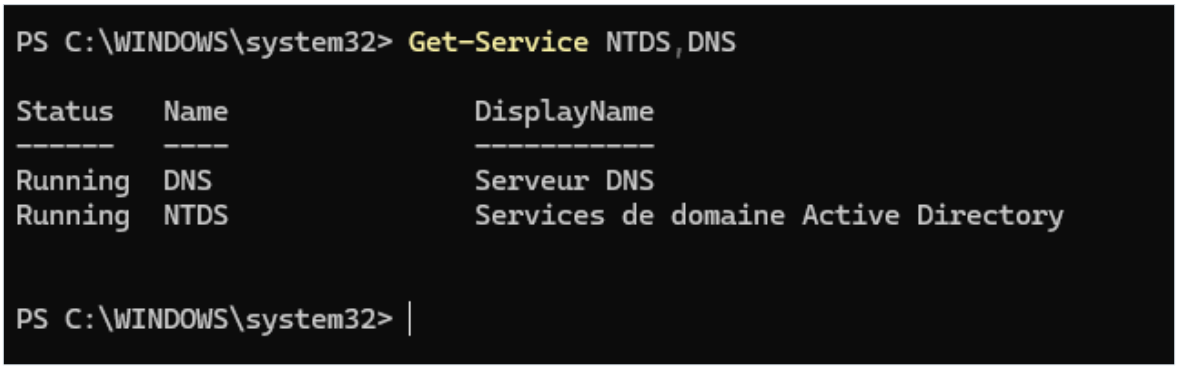
26. Maintenance Windows : résultats



F51. Windows Update affiche « Vous êtes à jour » après l'intervention. [PREUVES, p. 3]



F52. Historique des installations : correctifs de sécurité, .NET et composants Windows. [PREUVES, p. 4]



F53. Contrôle après intervention : NTDS et DNS restent Running. [PREUVES, p. 4]

Les captures montrent l'installation de correctifs et le maintien des services principaux du contrôleur. L'historique conserve les références installées, celles-ci décrivent le TP et ne constituent pas une liste de correctifs à appliquer sur un autre environnement.

L'état « à jour » correspond au moment de la vérification. Il doit être complété, lors d'une nouvelle maintenance, par une recherche de mises à jour et les tests DNS/AD habituels.

27. Chaîne de supervision

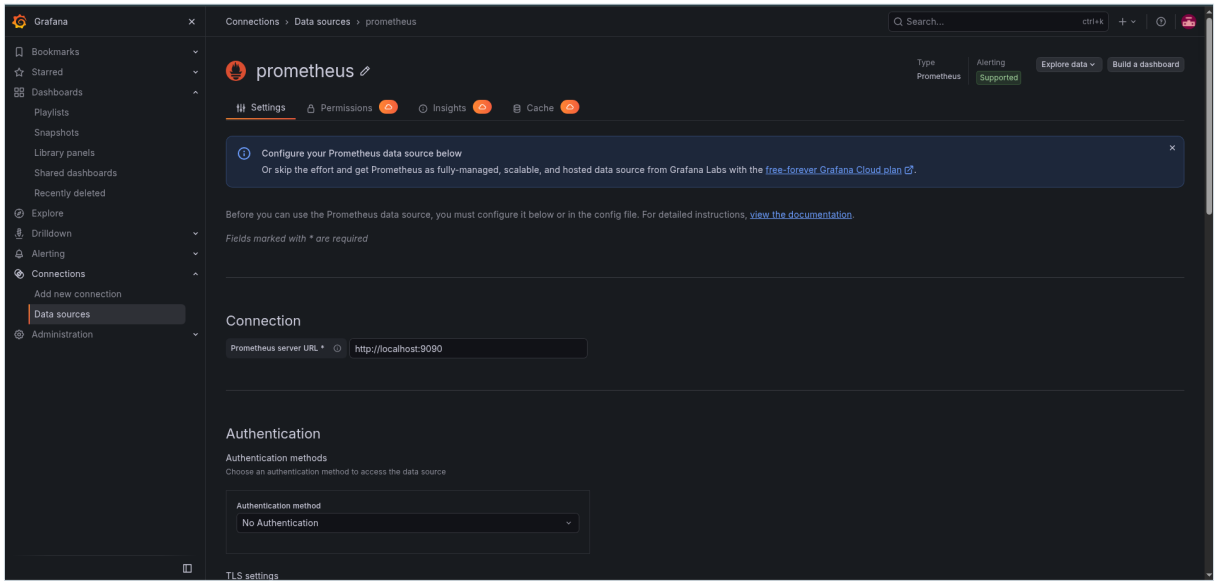
Node Exporter expose les métriques de SERVICE1 sur /metrics en TCP 9100, Windows Exporter expose celles d'AD en TCP 9182. Prometheus les collecte sur OPS1 et Grafana interroge Prometheus pour construire les graphiques.

```
(infraready) supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ curl -sS --max-time 10 -o /dev/null -w '
HTTP %{http_code}\n' http://192.168.50.20:9100/metrics
HTTP 200
(infraready) supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ curl -sS --max-time 10 -o /dev/null -w '
HTTP %{http_code}\n' http://192.168.50.10:9182/metrics
HTTP 200
(infraready) supervisor@srv-ops1-ubuntu26:~/infraready-ansible$
```

F54. Depuis OPS1, les endpoints Linux et Windows répondent tous deux HTTP 200. [OPS1, p. 6]

```
(infraready) supervisor@srv-ops1-ubuntu26:~/infraready-ansible$ curl -fsS --max-time 10 http://127.0.0.1
:3000/api/health
{
  "database": "ok",
  "version": "13.2.4",
  "commit": "0e79aef9d91abf3bdadae9af15763029072fb45b"
}(infraready) supervisor@srv-ops1-ubuntu26:~/infraready-ansible$
```

F55. L'API de santé Grafana répond avec database: ok et la version 13.2.4. [OPS1, p. 6]



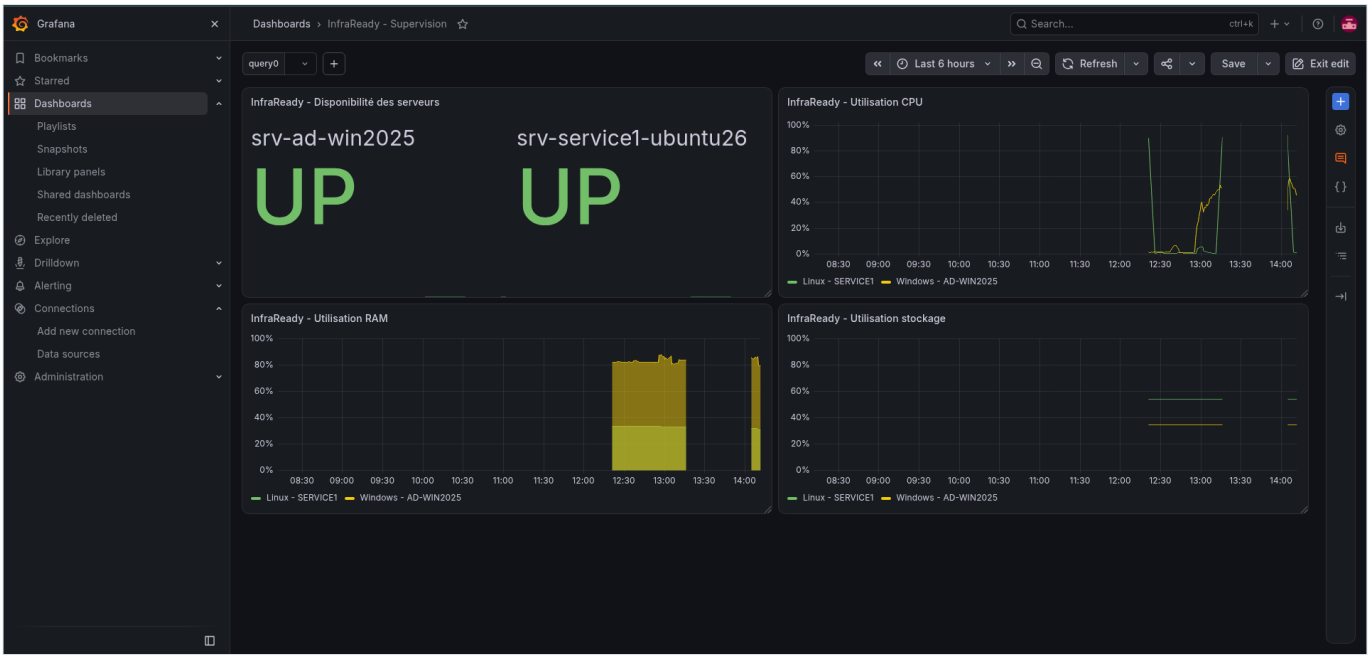
F56. Source Grafana Prometheus configurée vers http://localhost:9090 sur OPS1. [OPS1, p. 7]

Job versionné	Cible	Intervalle
linux	192.168.50.20:9100	15 s
windows	192.168.50.10:9182	15 s
prometheus	localhost:9090	5 s
node	localhost:9100 sur OPS1	15 s (valeur globale)

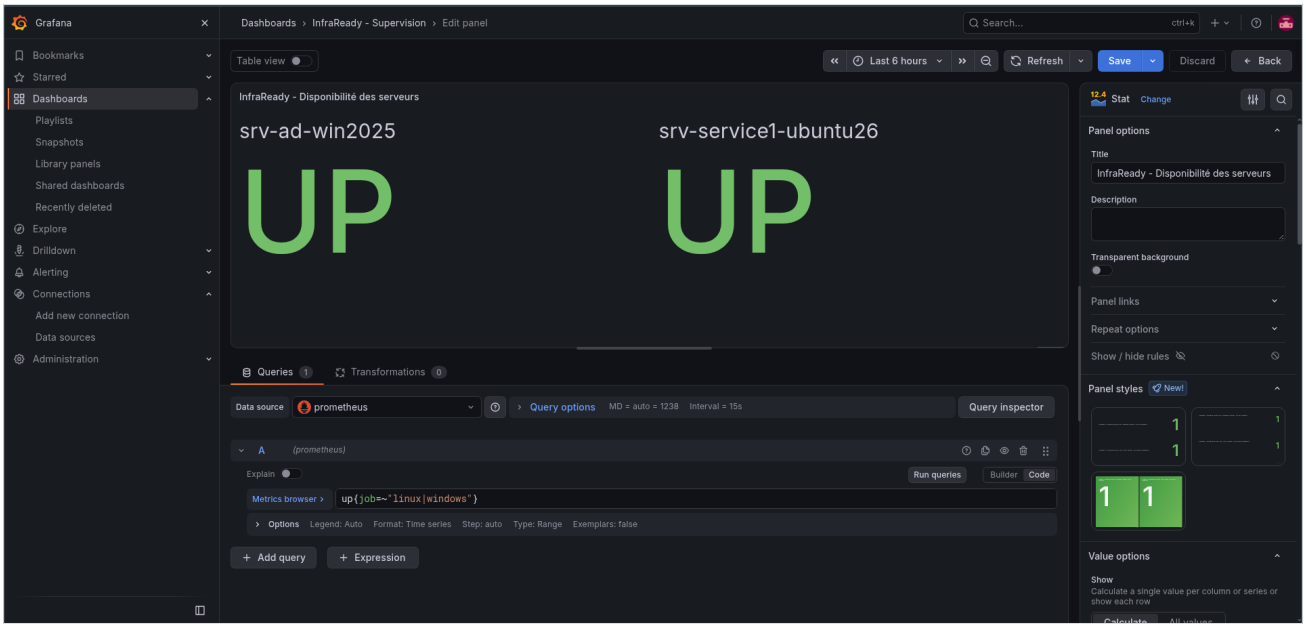
Les cibles locales sont configurées dans prometheus.yml, les captures démontrent la collecte Linux/Windows et le fonctionnement de Grafana.

28. Tableau de bord InfraReady

Le dashboard rassemble quatre panneaux : état de collecte, CPU, mémoire et stockage. Les légendes identifient SERVICE1 et AD. Les courbes visibles sont issues du laboratoire et non d'un jeu de données de démonstration.



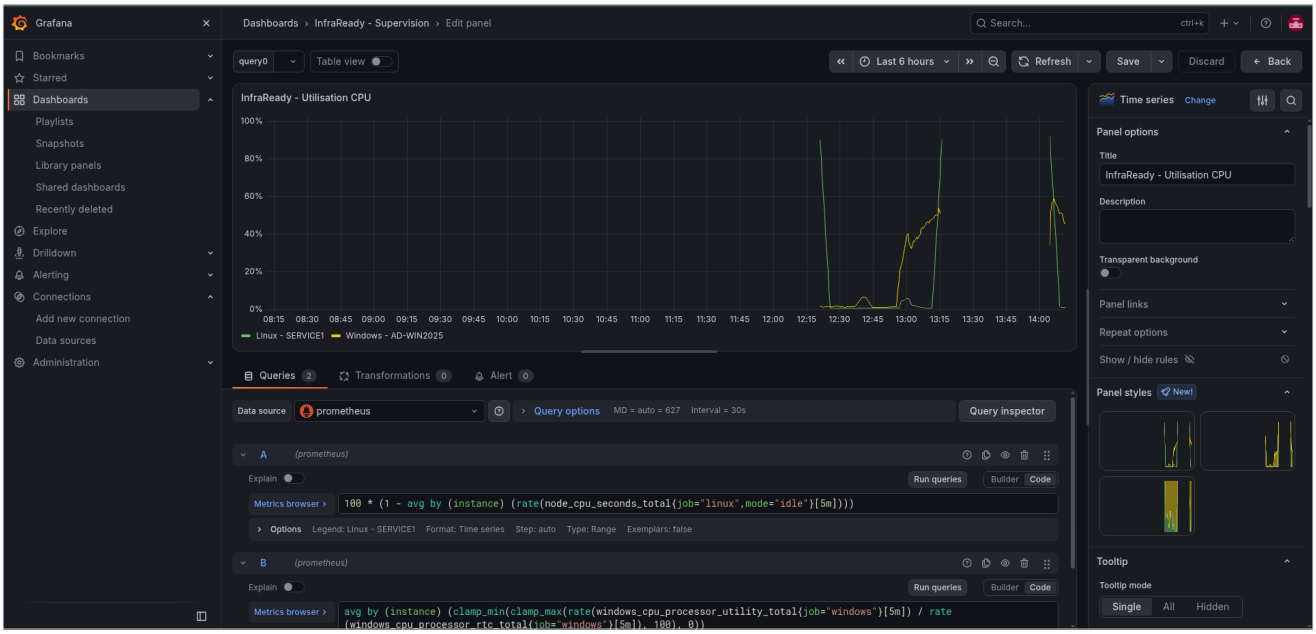
F57. Vue d'ensemble du tableau de bord « InfraReady - Supervision ». [OPS1, p. 9]



F58. Panneau de disponibilité : les deux cibles sont présentées UP. [OPS1, p. 7]

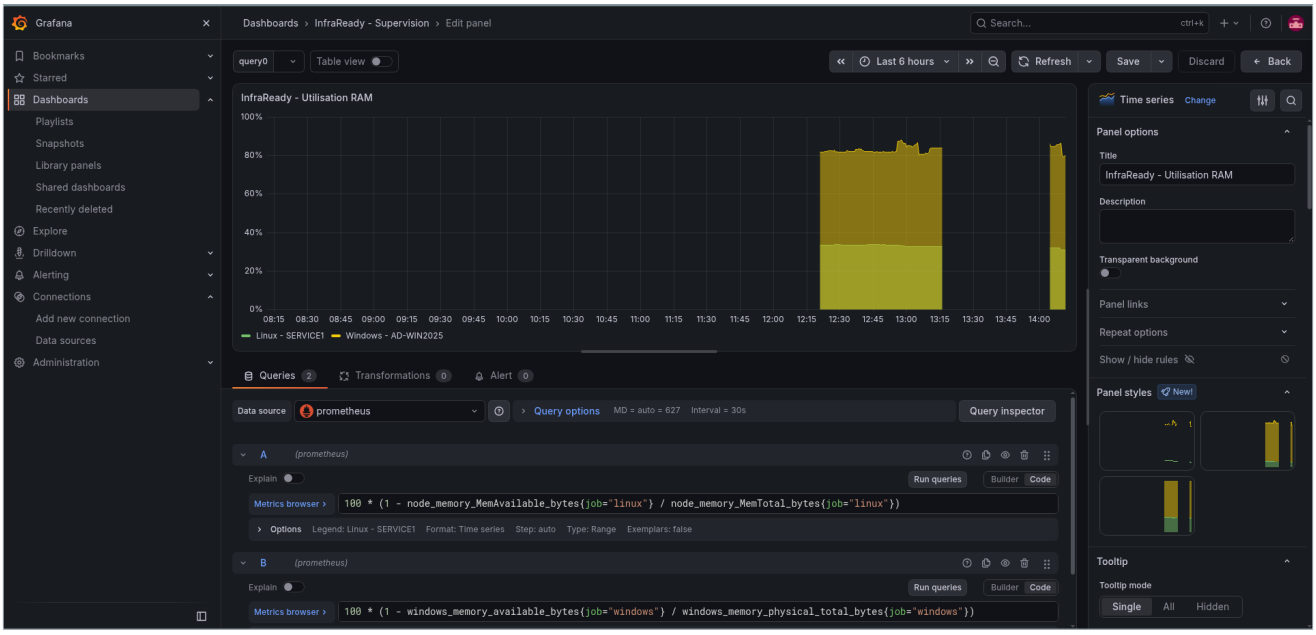
> **Sens de UP**
> La requête up{job=~"linux|windows"} retourne 1 quand la collecte Prometheus réussit et 0 lorsqu'elle échoue. L'export du dashboard associe 1 à UP et 0 à DOWN. Il faut un test HTTP distinct pour établir la disponibilité de Nginx. [R3]

29. Lecture des métriques CPU et mémoire



F59. CPU : les deux séries suivent l'activité Linux et Windows au cours du temps. [OPS1, p. 8]

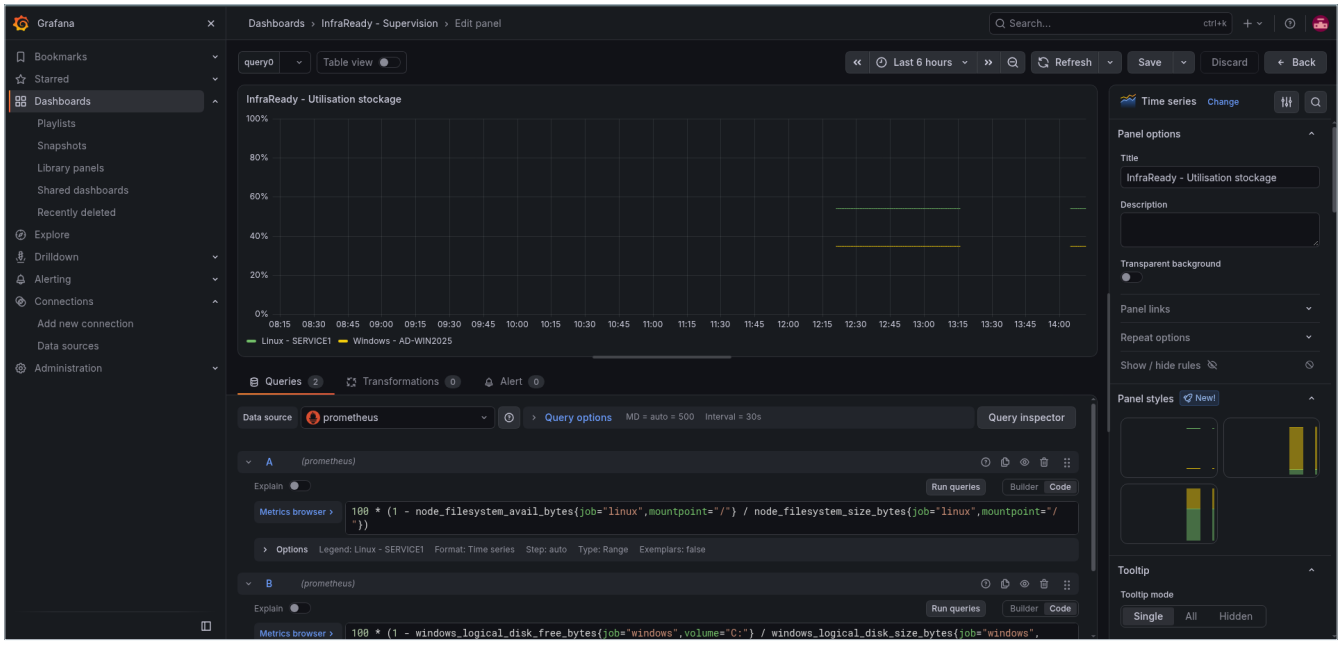
Linux : 100 × (1 - moyenne du taux idle sur 5 minutes). **Windows :** le dashboard utilise le rapport des taux windows_cpu_processor_utility_total et windows_cpu_processor_rtc_total, borné entre 0 et 100 puis moyenné par instance. La fenêtre de 5 minutes lisse les variations brèves.



F60. Mémoire : pourcentage calculé à partir de la mémoire disponible et de la mémoire totale. [OPS1, p. 8]

Pour chaque système, le calcul est **100 × (1 - mémoire disponible / mémoire totale)**. Linux utilise node_memory_MemAvailable_bytes et node_memory_MemTotal_bytes ; Windows utilise windows_memory_available_bytes et windows_memory_physical_total_bytes. Les valeurs décrivent chaque VM séparément, sans les additionner.

30. Stockage supervisé et règle d’alerte



F61. Stockage : comparaison de la racine Linux et du volume Windows C:. [OPS1, p. 9]

Le pourcentage est obtenu par 100 × (1 - espace disponible / capacité). Le filtre Linux porte sur **mountpoint="/"** ; Windows utilise **volume="C:"**.

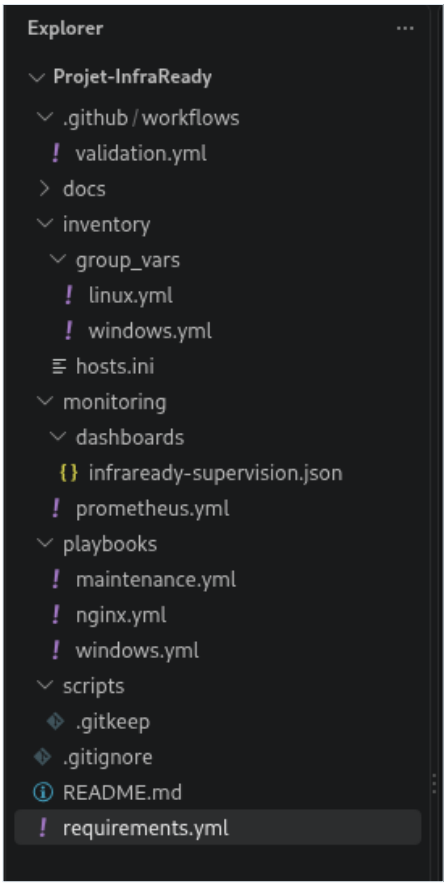
Alerte dans Grafana

Paramètre	Valeur visible sur les captures d’alerte
Nom	InfraReady - Serveur indisponible
Requête	up{job=~"linux|windows"}
Condition	Valeur inférieure à 1.
Évaluation / délai Pending	Toutes les 1 min / condition maintenue pendant 1 min.
Maintien après rétablissement	0 s, sévérité indiquée : warning.
Absence de données / erreur	États NoData / Error, distincts de la perte d’une cible connue.

L’incident des pages 38-41 démontre le passage Normal, puis Firing, puis Normal. La règle est gérée par Grafana et n’est pas présente dans l’export JSON du dashboard.

31. Dépôt GitHub et organisation

Dépôt du binôme : https://github.com/Leocollowald/Projet-InfraReady
Version consultée : commit **a42bafd3a6fd** du 23 octobre 2026.



F62. Organisation du dépôt [PREUVES, p. 5]

Organisation actuelle

inventory/hosts.ini
Hôtes Linux et Windows.

inventory/group_vars/
Variables de transport, Python et élévation.

playbooks/
nginx.yml, windows.yml et maintenance.yml.

monitoring/
Configuration Prometheus et export du dashboard.

.github/workflows/validation.yml
Contrôles exécutés par GitHub Actions.

requirements.yml
Collection ansible.windows en version 3.8.0.

docs/
Répertoire réservé à la documentation.

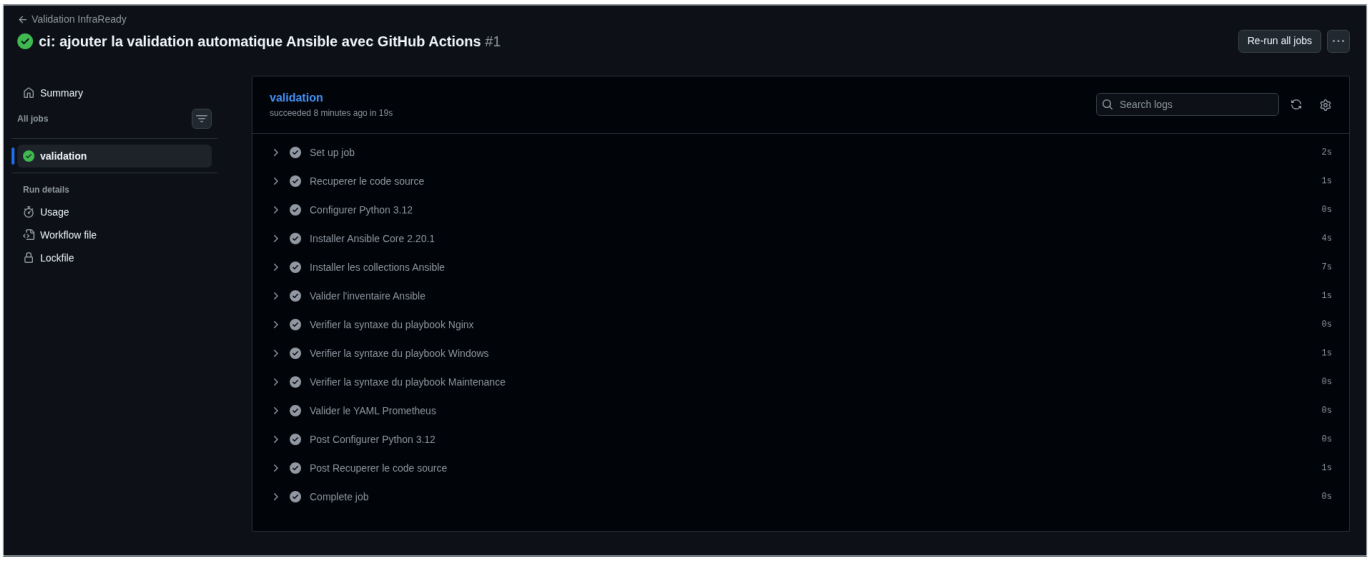
Reproduction à partir du dépôt

```
git clone https://github.com/Leocollowald/Projet-InfraReady.git
cd Projet-InfraReady
ansible-galaxy collection install -r requirements.yml
ansible-inventory -i inventory/hosts.ini --graph
```

Ces commandes doivent être exécutées sur un contrôleur ayant accès au laboratoire et possédant les dépendances de la page 18.
Le dépôt automatise des opérations ciblées, il ne provisionne pas les trois VM ni l'intégralité des services.

32. Validation continue avec GitHub Actions

Le workflow **Validation InfraReady** se déclenche sur push, pull_request ou lancement manuel. Il utilise un runner ubuntu-24.04, Python 3.12 et ansible-core 2.20.1. Les permissions du jeton sont limitées à contents: read et checkout désactive la persistance des identifiants.



F63. Exécution réussie du job validation : les contrôles sont verts, durée affichée de 19 s. [PREUVES, p. 5]

Contrôle automatique	Portée réelle
Installation des dépendances	Installation d'Ansible Core et des collections de requirements.yml.
Inventaire	ansible-inventory --graph ; un inventaire non interprétable entraîne un échec.
Playbooks	--syntax-check sur nginx.yml, windows.yml et maintenance.yml.
Prometheus	Analyse YAML avec yaml.safe_load, ce n'est pas une validation sémantique par promtool.
Exécution sur les VM	Aucune. Le runner GitHub n'accède pas directement au réseau NAT du laboratoire.

Contrôles à rejouer localement

```
ansible-playbook -i inventory/hosts.ini --syntax-check playbooks/nginx.yml
ansible-playbook -i inventory/hosts.ini --syntax-check playbooks/windows.yml
ansible-playbook -i inventory/hosts.ini --syntax-check playbooks/maintenance.yml
```

La CI confirme la cohérence syntaxique du dépôt. Les exécutions Ansible et les tests HTTP/DNS du TP restent nécessaires pour prouver le comportement réel des serveurs. Le workflow ne contient pas de déploiement automatique.

33. Sécurité et limites connues

Mesure / sujet	État documenté et limite
Périmètre	Trois VM du laboratoire sur réseau privé VMware NAT. Aucune intervention sur des systèmes externes n'entre dans le périmètre.
Administration Linux	Clé SSH dédiée, élévation demandée à l'exécution.
Administration Windows	WinRM limité à OPS1, Kerberos manuel, chiffrement des messages imposé et délégation désactivée. Le compte Administrateur est utilisé : un compte dédié avec délégation limitée serait préférable.
Secrets	Aucun mot de passe ni clé privée n'apparaît dans les fichiers Ansible consultés.
Droits	Séparation des droits par groupes AD et tests Linux autorisé/refusé.
Filtrage des métriques	La réponse HTTP est prouvée, le NAT seul ne remplace pas un pare-feu local.
Reproductibilité	Les playbooks et le dashboard sont versionnés. Le déploiement des VM, la création AD et la règle d'alerte Grafana ne sont pas automatisés par ces fichiers.
Supervision	Linux et Windows sont démontrés. Ajouter le volume /srv/infraready et un contrôle HTTP Nginx.

34. Preuves : infrastructure et administration

Cette matrice relie les résultats aux critères A, B et C du sujet. « Démontré » signifie que le résultat figure dans une capture du TP ou dans un fichier consulté.

Test / critère	Résultat constaté	Preuve
T01 - Adressage et réseau (A)	Trois adresses /24 documentées, paramètres IPv4 Windows visibles.	Inventaire p. 5 ; F03, p. 7
T02 - Connectivité (A)	OPS1 joint AD et SERVICE1, SERVICE1 joint AD ; 0 % de perte.	Démontré : F05, p. 8 ; F06, p. 8
T03 - DNS (A/B)	Le contrôleur est résolu en 192.168.50.10, SRV LDAP et Kerberos présents.	Démontré : F08, p. 9 ; F36, p. 21
T04 - Annuaire (B)	Domaine infraready.test, six utilisateurs et trois groupes de sécurité.	Démontré : F09, p. 10 ; F10, p. 10
T05 - Droits NTFS (B)	FullControl / Modify / ReadAndExecute différenciés par groupe.	ACL visible : F11, p. 11
T06 - Journaux Windows (B)	Événements System / Service Control Manager 7036 consultés.	Démontré : F12, p. 11
T07 - Service web (C)	Nginx répond 200 OK localement et depuis OPS1.	Démontré : F13, p. 12 ; F14, p. 12
T08 - Stockage (C)	Second disque 10 Go, GPT, partition ext4 et montage /srv/infraready.	Démontré : pages 13-15.
T09 - Montage persistant (C)	UUID dans fstab, vérification sans erreur après rechargement.	Configuration prouvée : pages 15-16 ; contrôle après démarrage à joindre.
T10 - Comptes et droits Linux (C)	ir_test membre d'infraready, écriture et lecture autorisées, lecture refusée à supervisor.	Démontré : F26, p. 17 à F28, p. 17
T11 - Journaux Linux (C)	Cycle de démarrage/arrêt Nginx consulté dans journalctl.	Démontré : F25, p. 16

35. Preuves : automatisation et exploitation

Test / critère	Résultat constaté	Preuve
T12 - Inventaire (D)	Groupes linux/windows, inventaire courant et variables lus dans le dépôt.	Démontré : F30, p. 19 ; dépôt [S8].
T13 - Connexions (D)	ping Linux et win_ping Windows retournent SUCCESS/pong.	Démontré : F31, p. 19 ; F32, p. 19
T14 - Playbook Linux (D)	Premier passage changed=1, second changed=0 ; failed=0 et unreachable=0.	Démontré : F34, p. 20 ; F35, p. 21
T15 - Playbook Windows (D)	Cinq tâches réussies, état DNS vérifié ; changed=0.	Démontré : F39, p. 23
T16 - Maintenance Linux (E)	Exécution réussie, Nginx et exporter actifs, aucun reboot-required.	Démontré : page 26.
T17 - Maintenance Windows (E)	Correctifs installés, état à jour, NTDS et DNS Running après intervention.	Démontré : pages 27-28
T18 - Collecteurs (F)	Endpoints Linux/Windows accessibles et séries présentes dans Grafana.	Démontré : F54, p. 29 ; F57, p. 30
T19 - Dashboard (F)	Disponibilité, CPU, RAM et stockage pour SERVICE1 et AD.	Démontré : pages 30-32 ; OPS1
T20 - Alerte / incident (F/H)	Arrêt Node Exporter, alerte Firing, relance et retour Normal.	Démontré : pages 38-40.
T21 - GitHub et CI (G)	Dépôt structuré, déclencheurs push/PR, workflow de validation réussi.	Démontré : pages 33-34, commit 1a96f4f6d0fb pour la CI.
T22 - Sécurité et documentation	WinRM filtré, chiffrement imposé dans le dépôt, limites et procédures explicitées.	Pages 24, 25, 35 et rapport d'incident.

Portée de la recette
Les preuves couvrent les principaux résultats demandés. Une exécution CI valide la syntaxe et les fichiers, elle ne remplace pas les tests fonctionnels des VM. Les limites de sécurité, de couverture de supervision et de reproductibilité sont conservées page 35.

36. Incident : perte des métriques Linux

Cible : SRV-SERVICE1-UBUNTU26 (192.168.50.20).
Nature : incident volontaire, limité au laboratoire et réversible. **État final :** collecte rétablie, alerte revenue à Normal.

Symptôme et impact

Après l'arrêt de prometheus-node-exporter, Prometheus ne peut plus collecter les métriques de SERVICE1. L'indicateur up de cette cible passe à 0 et l'alerte Grafana se déclenche. La cible Windows reste à 1. L'impact démontré est la perte de visibilité sur les métriques Linux, aucune panne de Nginx ou extinction de la VM n'est établie.

```
supervisor@srv-service1-ubuntu26:~$ sudo systemctl stop prometheus-node-exporter
systemctl is-active prometheus-node-exporter
[sudo: authenticate] Password:
inactive
supervisor@srv-service1-ubuntu26:~$
```

F64. Injection contrôlée : arrêt de prometheus-node-exporter, puis état inactive. [OPS1, p. 10]

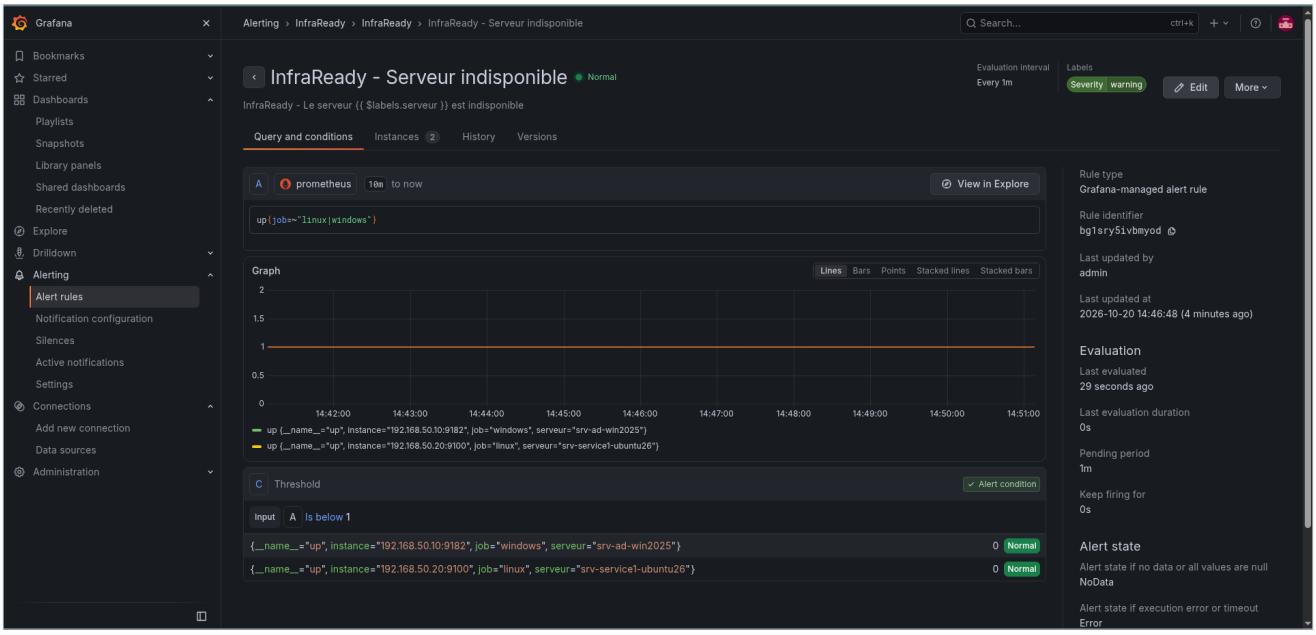
```
supervisor@srv-service1-ubuntu26:~$ sudo journalctl -u prometheus-node-exporter --since "2026-10-20 12:5
0" --until "2026-10-20 13:13" --no-pager | grep -E 'Stopping|Stopped|Started'
oct. 20 12:51:55 srv-service1-ubuntu26 systemd[1]: Stopping prometheus-node-exporter.service - Prometheu
s exporter for machine metrics...
oct. 20 12:51:55 srv-service1-ubuntu26 systemd[1]: Stopped prometheus-node-exporter.service - Prometheus
 exporter for machine metrics.
oct. 20 13:11:46 srv-service1-ubuntu26 systemd[1]: Started prometheus-node-exporter.service - Prometheus
 exporter for machine metrics.
supervisor@srv-service1-ubuntu26:~$
```

F65. Journaux systemd : arrêt à 12:51:55 et redémarrage à 13:11:46 le 20 octobre. [OPS1, p. 11]

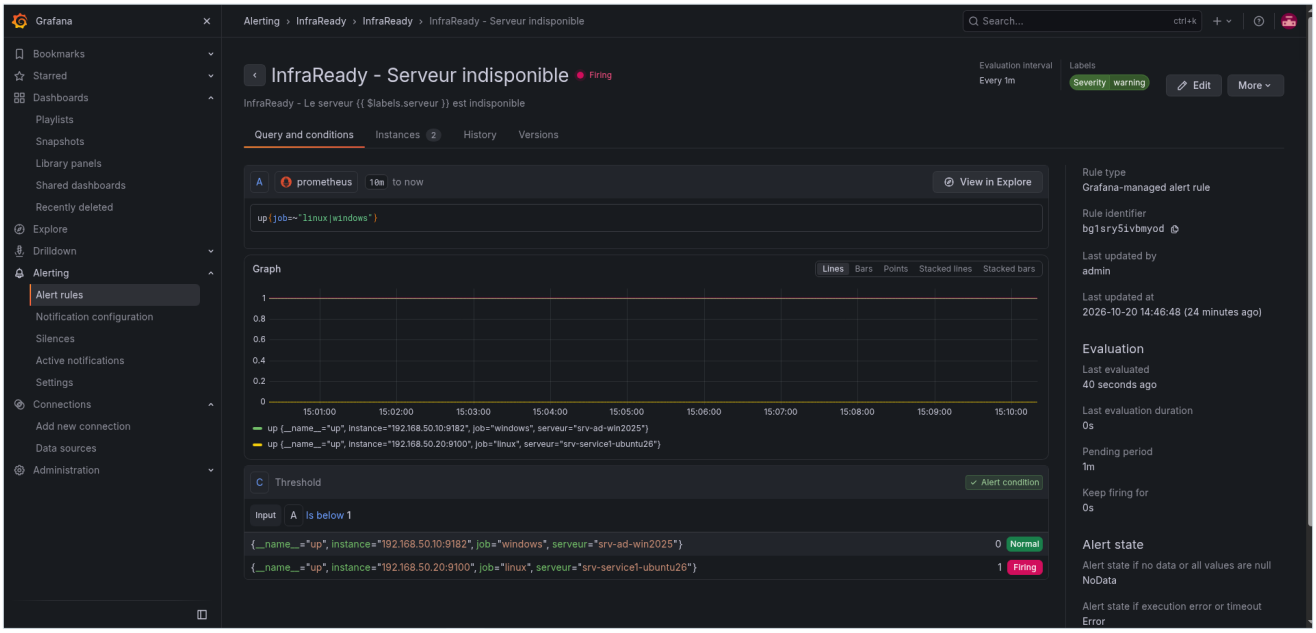
Heure du journal	Événement établi	Interprétation
12:51:55	Stopping puis Stopped	Le service de collecte local est arrêté.
Pendant l'arrêt	up Linux = 0, alerte Firing	La collecte échoue, l'instance Windows reste normale.
13:11:46	Started	Le service Node Exporter est relancé.
Après relance	active, HTTP 200, alerte Normal	Le service et la chaîne de collecte fonctionnent de nouveau.

37. Détection et qualification de l'alerte

La règle surveille les deux jobs linux et windows avec le seuil up < 1. Une évaluation par minute et un délai Pending d'une minute évitent de déclencher immédiatement sur un seul résultat transitoire. L'état Firing visible constitue la preuve du déclenchement.



F66. Situation initiale : deux cibles à 1, règle et instances à Normal. [OPS1, p. 10]



F67. Pendant l'incident : série Linux à 0 et instance Linux Firing ; Windows reste à 1. [OPS1, p. 10]

38. Correction et retour à la normale

Cause retenue et action

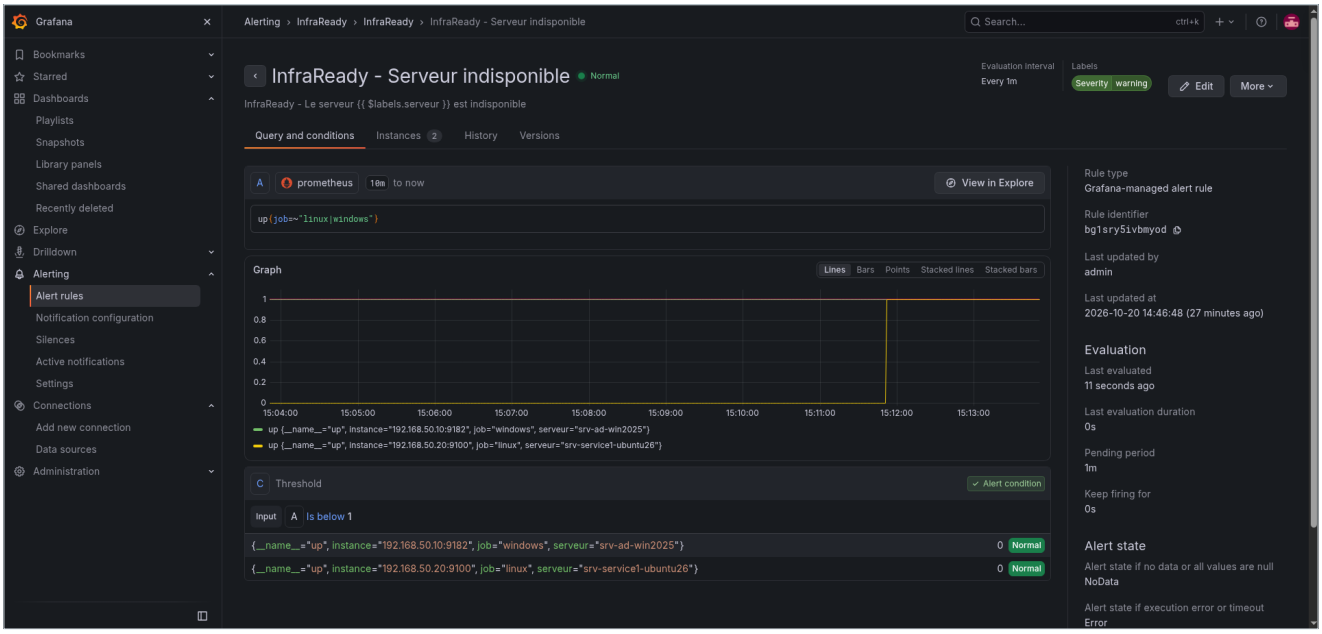
La cause est l'arrêt volontaire du service Node Exporter sur SERVICE1. Elle est établie par la commande d'arrêt, l'état inactive et les événements systemd. La correction consiste à démarrer ce même service, puis à vérifier successivement son état, son endpoint et la supervision distante.

```
supervisor@srv-service1-ubuntu26:~$ sudo systemctl start prometheus-node-exporter

systemctl is-active prometheus-node-exporter

curl -fsS http://localhost:9100/metrics -o /dev/null -w 'HTTP %{http_code}\n'
[sudo: authenticate] Password:
active
HTTP 200
supervisor@srv-service1-ubuntu26:~$
```

F68. Correction : systemctl start ; le service devient active et /metrics répond HTTP 200. [OPS1, p. 11]



F69. Grafana après correction : reprise de la série Linux et retour des instances à Normal. [OPS1, p. 11]

Vérification	Résultat	Niveau validé
systemctl is-active prometheus-node-exporter	active	Processus géré par systemd démarré.
curl sur localhost:9100/metrics	HTTP 200	Endpoint de métriques disponible localement.
Grafana : up et état de règle	Linux revenu à 1, Normal	Collecte Prometheus et évaluation Grafana rétablies.

Le retour à la normale est confirmé à plusieurs niveaux : relance locale, réponse applicative et récupération des métriques. Aucune modification de DNS, de pare-feu ni de configuration Prometheus n'a été nécessaire dans cet incident.

39. Bilan du diagnostic et prévention

Démarche suivie

L'alerte attire l'attention sur la cible Linux. Le contrôle systemctl révèle un collecteur arrêté, journalctl confirme les événements. La relance rend l'endpoint accessible, puis la disparition de l'alerte confirme la reprise de la chaîne de supervision. La corrélation entre signal distant et état local permet ici d'isoler la cause sans modifier les autres services.

Question de diagnostic	Réponse établie
Le serveur est-il nécessairement arrêté ?	Non. up=0 signifie que la collecte échoue. La session de terminal sur SERVICE1 reste utilisable.
L'incident est-il global ?	La capture conserve Windows à up=1, l'échec observé est limité à la cible Linux.
Une cause réseau/DNS est-elle démontrée ?	Non. Le scénario et l'état du service expliquent l'incident. Aucun changement réseau n'est montré.
Le redémarrage du service suffit-il ?	Oui pour ce scénario : active, HTTP 200 puis Normal montre le rétablissement.